FULL PRODUCT + USER EXPERIENCE SPECIFICATION

# ShipSense

A trust-first, AI-assisted CI/CD Management Platform connecting Version Control (GitHub), CI Pipelines (Jenkins, GitHub Actions), Cloud Deployments (Vercel, Netlify, Railway, Render), Observability (Grafana, OpenTelemetry), Alerts (Slack), and Project Management (Jira)

| | |
|---|---|
| **Document** | Full product requirements, detailed features, UI states, and end-to-end application flows |
| **Build intent** | Complete portfolio product concept; implementation is intentionally specified separately |
| **Primary users** | Platform engineers, SREs, developers, security engineers, and engineering leaders |
| **Signature loop** | Alert or failed run -> evidence -> hypothesis -> governed PR -> verification -> replay |
| **Research date** | 23 July 2026 |
| **Status** | Full product specification v1.0 |

**BUILD DECISION**
Proceed with the full ShipSense portfolio project, delivered in coherent product releases. Do not claim the broad platform is unique. The strongest product thesis is an inspectable evidence-to-PR-and-replay workflow with hard approval, policy, verification, and replay boundaries.

This specification covers the complete project-centric delivery lifecycle and the full target product. Release 1 proves the core experience by unifying Version Management (GitHub), CI Pipelines (Jenkins, GitHub Actions), Managed Cloud Deployments (Vercel, Netlify, Railway, Render), Observability (Grafana, OpenTelemetry), Alerts (Slack), and Project Management (Jira) under a single project workspace where AI assists in managing the entire CI/CD plane.

## What this document is

This is the complete product, feature, interface, and application-flow specification for ShipSense. It describes what users see and do, how states and permissions behave, how the product handles success and failure, and how the full product is validated and released.

**INTENTIONAL BOUNDARY**
Implementation technicalities—code structure, APIs, backend services, infrastructure topology, database design, and deployment configuration—are intentionally excluded. They belong in a separate engineering specification maintained with the code.

# Executive brief

ShipSense is a project-centric, AI-assisted CI/CD management platform and operations control center. Its broad experience unifies version management (GitHub), build and CI automation (Jenkins, GitHub Actions), cloud deployments (Vercel, Netlify, Railway, Render), observability (Grafana, OpenTelemetry), project management (Jira), alerts (Slack), security, releases, incidents, an application-scoped AI Operations Copilot, and engineering intelligence; its strongest product thesis is a transparent evidence-to-remediation loop across every project.

## The one-sentence product

**PRODUCT DEFINITION**
ShipSense helps software teams ask questions across every authorized application, understand delivery and runtime change through live cited evidence, coordinate releases and incidents, investigate failures, and create governed, verified remediation pull requests—without hiding uncertainty or bypassing existing approvals.

## Research-backed decision

Proceed as a full portfolio product. The category is real and valuable, but the broad platform concept is not novel: CloudBees, Port, AWS, Azure, Datadog, and Harness already cover meaningful portions of the control-plane, investigation, remediation, and verification space. [S10][S11][S13][S15][S17][S19] ShipSense should therefore demonstrate exceptional integration, product judgment, operational safety, and evaluation discipline rather than claim that no competitor exists.

| Dimension | Assessment | Rationale |
|---|---|---|
| Problem importance | 4.5 / 5 | Delivery and operations remain fragmented; CI/CD interoperability and platform quality matter. [S04][S08] |
| Broad product novelty | 2.0 / 5 | Several mature products already unify delivery, catalog, observability, incidents, and AI actions. |
| Wedge differentiation | 3.5 / 5 | Versioned evidence, contradictions, abstention, governed PRs, and replay can be an unusually inspectable product experience. |
| Portfolio/CV value | 5.0 / 5 | The project demonstrates product research, dense operational UX, distributed workflows, integrations, AI safety, evaluation, security, and SRE thinking. |
| Solo full-scope feasibility | 2.0 / 5 | Full breadth is large; coherent releases and one reference integration pack make progress demonstrable. |
| Market confidence today | 2.5 / 5 | Desk research is strong; customer interviews, prototype tasks, and a design-partner pilot are still required. |

### Full-product outcome

- A platform engineer can navigate from organization fleet to any service, release, pipeline, workload, SLO, incident, finding, proposal, or audit decision without losing environment and time context.
- Any authorized user can view and manually operate every connected tool (GitHub, Jenkins, Vercel/Netlify/Railway/Render, Grafana/OTel, Slack, Jira) directly from the UI, or instruct the AI Copilot to execute operational actions, manage pipelines, trigger deployments, update Jira issues, and generate verified remediation PRs.
- An on-call responder can move from alert to a reviewed evidence pack, multiple hypotheses, and a governed remediation proposal while seeing contradictions, gaps, and abstention.
- A developer can understand why a change failed, what artifact is being promoted, how environments differ, and whether a proposed fix is minimal and verified.
- A security engineer can trace an exact artifact to findings, SBOM, signature, provenance, policy, exception, release, and outcome.
- An engineering leader can inspect delivery, reliability, quality, and automation trends without using individual productivity rankings or opaque scores.

### Document map

| Part | Contents |
|---|---|
| 1–3 | Research validation, product strategy, users, roles, permissions, and product boundaries. |
| 4–6 | Complete CI/CD lifecycle, capability map, information architecture, and shared experience rules. |
| 7–8 | Detailed feature specifications and screen-by-screen UI blueprints. |
| 9–12 | Complete application flows, edge cases, AI copilot, AI trust, governed autonomy, and evaluation. |
| 13–16 | Notifications, collaboration, analytics, product requirements, and integrations. |
| 17–19 | Releases, validation, risks, prerequisites, acceptance/readiness, research sources, and glossary. |

## 1. Evidence, novelty, and market position

The product should be ambitious in scope and conservative in claims. This chapter separates verified market evidence from the product hypothesis.

### Why the problem matters

DORA's 2025 research reports near-universal AI use among respondents while emphasizing that AI amplifies existing organizational strengths and weaknesses; the same research reports positive relationships with throughput/product outcomes alongside negative stability effects. [S01][S02] DORA

also reports wide platform adoption and dedicated platform teams, reinforcing the value of an integrated internal product experience—but not proving demand for ShipSense specifically. [S02][S04]

The CD Foundation reports broad DevOps participation and continued use of multiple tools, making interoperability a recurring product problem. [S08] ShipSense should not try to replace the execution engines that teams already trust. It should make their combined delivery state, evidence, decisions, and outcomes understandable.

## Competitive reality

| Competitor | Verified adjacent capability | Implication | ShipSense response |
|---|---|---|---|
| CloudBees Unify | CI, application security, feature management, analytics, release orchestration, and multi-tool integrations. | Established enterprise DevOps control plane with broad lifecycle management. | Focus on inspectable evidence, transparent contradiction tracking, replay labs, and strict PR governance rather than broad ecosystem ownership. [S10] |
| Port | Software catalog, developer self-service, engineering intelligence, conversational catalog assistant, contextual agent widgets, and CI/CD failure remediation workflows. | Direct platform comparator with strong internal developer portal (IDP) traction and embedded AI actions. | Differentiate through application-scoped context capsules, cross-plane evidence ledgers, explicit disconfirming evidence, governed remediation PRs, and versioned replay. [S11][S12][S61] |
| AWS DevOps Agent | Context-aware chat, release readiness, automated testing, incident investigation, topology questions, and proactive incident prevention. | Cloud-native autonomous agent with deep operational context across AWS infrastructure and delivery pipelines. | Maintain strict provider neutrality across heterogeneous tools (Jenkins, Vercel, Netlify, Railway, Render, Grafana, Jira) while delivering portable evidence contracts. [S13][S14][S60][S63][S64] |
| Azure SRE Agent | Chat over live logs, source code, and Azure resources; automated alert investigation, code fixes, branches/PR creation, and health verification. | Direct incumbent comparator for end-to-end incident-investigation and code-remediation workflows. | Emphasize multi-cloud/multi-tool scope, dynamic least-data source plans, deterministic policy gates, and reproducible evaluation benchmarks. [S15][S16][S68] |
| Datadog Bits AI | Observability-native chat, telemetry exploration, automated incident investigation, code fixes, detection, and AI agents. | Strong telemetry context with immense incumbent distribution and deep observability hooks. | Use observability data as input evidence while maintaining explicit links to version control, CI DAGs, artifacts, GitOps, policy gates, and human approvals. [S17][S18][S62] |
| Harness | AI-assisted software delivery, AI SRE, continuous deployment verification, and policy governance. | Dominant enterprise incumbent in automated delivery governance and deployment verification. | Differentiate via transparent evidence manifests, disconfirmation tracking, and a frozen replay lab for regression evaluation. [S19][S20][S21] |
| incident.io and Rootly | Incident summaries (Scribe), live-call notes, catch-up briefs, suggested updates, shift handoffs, deploy checks, and automated retrospectives. | Market leaders in incident orchestration, conversational workflows, and AI documentation assistants. | Connect the incident narrative directly to version control, CI/CD pipelines, artifact digests, cloud deployment states, and observed recovery without replacing human command. [S65][S66][S67] |

### Novelty verdict

**VALIDATION STATUS**
Broad novelty: low. Product value: plausible. Portfolio value: very high. Differentiation is a hypothesis, not a fact, until target users prefer the evidence/contradiction/replay experience in comparative tasks and pilots.

### The defensible product thesis

- Evidence before answer: every important claim opens the exact source, time, scope, and freshness.
- Contradictions are first-class: ShipSense must show what weakens a hypothesis, not only what supports it.
- Recommendation is separated from authorization: probabilistic reasoning cannot grant its own permission.
- Remediation is a governed proposal: minimal diff, deterministic checks, human approval, and observed outcome.
- Every outcome can become a frozen replay case so product/model/tool/policy changes are measurable.
- Dual-mode operational control (Manual + AI): Every connected integration exposes comprehensive views and direct manual controls alongside AI-assisted management, so teams have full manual command over their CI/CD plane while using AI to automate, orchestrate, and remediate.

### Working-name risk

ShipSense is a clear working name for a portfolio project, but it is not distinctive on the public web. Existing uses include shipping-email software, logistics products, a longstanding ShipWorks feature, and a recent product-intelligence project. [S56][S57][S58][S59] Treat the name as not legally cleared; conduct domain, company-name, repository, app-store, and professional trademark review before commercial launch.

## 2. Product strategy, principles, and boundaries

ShipSense is a complete product, but it remains a control and intelligence experience around existing delivery systems—not a replacement for every tool.

## Vision

Give every software team an AI-assisted, project-centric CI/CD management plane—providing a shared, trustworthy understanding of what code, builds (Jenkins/GitHub), deployments (Vercel/Netlify/Railway/Render), Jira tasks, and telemetry (Grafana/OpenTelemetry) exist, what is changing, what is failing, why it is likely failing, what actions are safe, and whether the action actually improved project and customer outcomes.

## Product principles

| Principle | Required behavior |
|---|---|
| Traceable by default | Every state, score, summary, cause, and decision links to source evidence and time. |
| Historical truth | Incident and release views reconstruct context at the relevant time instead of substituting today's topology. |
| Human authority | ShipSense recommends and proposes; eligible people and existing systems authorize material effects. |
| Uncertainty is a state | Unknown, stale, partial, contradictory, and abstained are explicit—not hidden behind green or a confident paragraph. |
| Progressive disclosure | Fleet summaries lead to service, release, runtime, evidence, and source detail without overwhelming first view. |
| Provider-neutral language | Users learn one state model while still being able to open the provider's raw object. |
| Safe product degradation | If ShipSense or a connector is impaired, external CI/CD continues and the UI becomes visibly stale/read-only. |
| Learning loop | Resolved outcomes improve runbooks, catalog, policies, tests, and replay cases. |
| Accessible operations | Keyboard, focus, semantics, contrast, non-color labels, reduced motion, and readable dense layouts are core requirements. |
| No vanity automation | Abstention and blocked unsafe actions are successes when evidence is inadequate. |

## Goals

- Unify user navigation and context across the complete delivery and operations loop.
- Cut time from failed signal to relevant evidence and safe next decision.
- Make project-wide artifact lineage, Jenkins/GitHub CI gates, preview and production environment differences, cloud deployment state (Vercel, Netlify, Railway, Render), Jira issue alignment, and runtime health legible.

- Deliver full dual-mode operational control: provide comprehensive views and direct manual action controls for every connected tool alongside AI-driven execution, pipeline management, deployment orchestration, and governed remediation.
- Turn operational outcomes into reusable product knowledge and evaluation cases.
- Demonstrate production-grade product judgment as a portfolio artifact.

## Explicit non-goals

- ShipSense is not a replacement CI runner, source-control host, cloud hosting provider, observability datastore, secret manager, ticketing system, or team-chat replacement. Its AI Operations Copilot is a governed interface that connects and manages across GitHub, Jenkins, Vercel/Netlify/Railway/Render, Grafana/OpenTelemetry, Slack, and Jira within a unified project boundary.
- While ShipSense provides direct manual and AI-assisted operational controls across connected CI/CD, deployment, and project management tools, AI-initiated high-risk production actions remain governed by deterministic policy and explicit human approval rather than unchecked autonomous mutation.
- The product does not promise perfect root-cause certainty, universal automatic fixes, or compliance certification.
- The product does not rank individual developers by output metrics.
- The full product does not require every integration to be present before partial value is visible; missing coverage must be explicit.

## Primary market and product assumptions

| Assumption | Working choice | Validation need |
|---|---|---|
| Initial customer | Software organizations with 20–500 engineers, several services, on-call practice, and CI/CD fragmentation. | Interview 12–15 platform/SRE leaders and individual responders. |
| Reference environment | GitHub Actions, Kubernetes, Argo CD/Rollouts, Prometheus/Grafana/OpenTelemetry, Slack. | Confirm this represents enough target workflows for first design partners. |
| Buyer | Head of platform/SRE/engineering productivity; security may co-sponsor. | Map budget, procurement, and replacement/augmentation story. |
| Daily user | Platform engineer, SRE/on-call, and developer. | Run task-based prototype studies for each role. |
| Trust posture | Read-first, PR-mediated effects, deterministic policy, explicit approvals. | Measure whether users prefer control over faster autonomy. |

| Assumption | Working choice | Validation need |
|---|---|---|
| Deployment preference | Hosted product with private/customer-side integration option as enterprise scope. | Test data-residency and access barriers in discovery. |

## 3. Users, roles, and permissions

ShipSense serves multiple roles in the same operational context. Role-specific defaults reduce noise, while shared objects preserve collaboration.

### Personas

| Persona | Job | Default information | Typical authority |
|---|---|---|---|
| Platform engineer | Own the golden paths, clusters, GitOps controllers, policies, and connectors. | Fleet health; catalog coverage; policy drift; connector status; automation outcomes. | Configure connectors, policies, templates, environments, and non-production actions. |
| Site reliability engineer / on-call | Restore service safely and build reusable operational knowledge. | Incidents, causal evidence, recent changes, dependencies, SLO burn, safe next actions. | Investigate, propose or reject actions, request approvals, resolve incidents, author runbooks. |
| Application developer | Ship changes and understand why a build, deploy, or service is unhealthy. | PR checks, pipeline DAG, artifact lineage, environment state, logs/traces, remediation PRs. | Retry safe jobs, open or edit PRs, compare environments, run non-production verification. |
| Security engineer | Enforce software supply-chain controls and investigate exposures. | SAST/SCA/secrets/IaC results, SBOM, signature, provenance, exceptions, policy decisions. | Manage security policy, approve time-bound exceptions, inspect attestations and audit history. |
| Engineering leader | Improve delivery outcomes without rewarding unsafe volume. | DORA metrics, SLOs, incident trends, rework, flaky tests, cost, adoption, automation quality. | Read analytics and drill down; no operational write access by default. |
| Organization administrator | Control tenancy, identity, retention, billing, and enterprise integrations. | SSO/SCIM, roles, data residency, audit export, usage, quotas, connector health. | Manage tenant configuration and emergency access with audited break-glass. |

### Role experience rules

- Permissions are visible at the action point; disabled controls explain the missing role, provider scope, policy, or approval.
- Every connected integration supports both direct manual operation and AI-assisted control subject to RBAC and provider permissions; provider permission remains authoritative for provider-owned effects.

- Production approval uses separation of duties: the proposer cannot be the sole approver.
- Leadership analytics are aggregated and drillable but never used to score individuals.
- Security data may be scoped more narrowly than general delivery data; redaction preserves event existence and audit.
- Break-glass is time-bound, reauthenticated, separately notified, limited in scope, and reviewed afterward.

## Permission matrix

| Capability | Platform | Developer | Security | SRE | Leader | Administrator |
|---|---|---|---|---|---|---|
| View fleet/catalog/delivery/runtime | Yes | Yes | Yes | Yes | Aggregate | Tenant |
| Cancel/retry CI run | Policy | Yes | No | Yes | No | Configure |
| Edit service ownership/runbook/SLO | Yes | Yes | Policy | Yes | No | Configure |
| Create/coordinate incident | Yes | Yes | View | Yes | View | Configure |
| Use AI operations copilot | Yes | Yes | Scoped | Yes | Aggregate | Configure/disable |
| Run AI investigation | Yes | Yes | View | Yes | No | Configure/disable |
| Publish AI-generated durable knowledge | Review | Own service | Review | Review | No | Configure |
| Generate/edit remediation proposal | Yes | Yes | Review | Yes | No | Configure |
| Approve production proposal/release | No if proposer | Policy | Security scope | Policy | No | Configure matrix |
| Merge PR | External Git permission | External Git permission | No | External Git permission | No | No |
| Hold/resume/abort rollout | Policy | Policy | No | Yes | No | Configure |
| Approve security exception | No | No | Yes | Policy | No | Configure matrix |
| Publish production policy | Review | No | Yes | Yes | No | Configure matrix |
| View audit | Scoped | Own actions | Yes | Yes | Aggregate | Yes |
| Manage connectors/SSO/retention | No | No | Review | No | No | Yes |
| View analytics | Scoped | Team | Scoped | Scoped | Yes | Yes |
| Export/delete tenant | No | No | Review | No | No | Two-admin rule |

### Approval model

| Action class | Default rule | Approval invalidated by |
|---|---|---|
| Read/investigate | Allowed within role and connected-source scope. | Loss of source permission or tenant/service scope. |
| Retry/cancel CI | Allowed when provider permits and policy classifies action as bounded. | Run superseded, state changed, quota or policy changed. |
| Draft remediation PR | Proposal review plus allowed repository/path; provider merge permission remains external. | Base commit, diff, verification, policy, or approver changes. |
| Promote production release | Required checks plus eligible human approval and change-window policy. | Artifact/evidence/config/dependency/policy material change or expiry. |
| Abort rollout | May be pre-authorized for named SRE roles when a documented threshold fails. | State no longer active; competing recovery in progress. |
| Security exception | Security approver, service owner, scope, rationale, compensating control, and expiry. | Finding/artifact/policy/scope changes or expiry. |
| Publish policy | Separate author and production reviewer; staged impact preview. | Draft change after review or conflict with newer policy. |
| Tenant deletion/break-glass | Step-up authentication and two-admin or emergency rule. | Timeout, changed scope, legal hold, or cancellation. |

## 4. Complete software-delivery lifecycle

ShipSense spans the complete project-centric CI/CD management loop from Jira project planning and GitHub version control through Jenkins builds, Vercel/Netlify/Railway/Render deployments, Grafana/OpenTelemetry observability, Slack alerting, and AI-assisted remediation. It links and manages stages across your project; it does not replace the specialized tools that execute them.

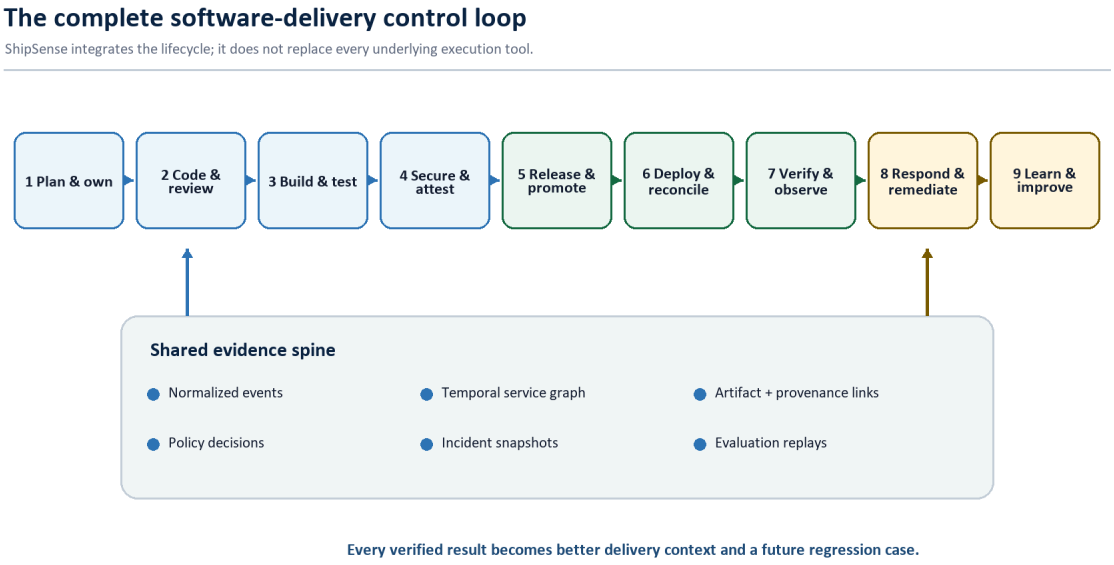



*Figure 1. Complete ShipSense software-delivery control loop.*

| Stage | ShipSense experience | Visible outcome |
|---|---|---|
| 1. Plan and own | Service ownership, tier, dependencies, SLOs, policies, templates, runbooks. | Catalog record and release policy. |
| 2. Code and review | Branch, commit, PR, review, checks, change classification, affected-service detection. | Reviewable change with traceable intent. |
| 3. Build and test | Reproducible build, lint, unit/integration/E2E/contract tests, coverage, cache, runner health. | Immutable candidate artifact and test evidence. |
| 4. Secure and attest | SAST, SCA, secrets, container/IaC/license checks, SBOM, signature, provenance, policy. | Verified artifact plus policy decision. |
| 5. Release and promote | Version, notes, environment rules, approvals, change window, dependency order, feature flags, DB plan. | Approved release candidate. |
| 6. Deploy and reconcile | GitOps PR/merge, sync, hooks/waves, controller progress, drift, rolling/canary/blue-green. | Observed desired and actual state. |
| 7. Verify and observe | Probes, smoke tests, analysis runs, telemetry, SLO/error budget, business KPI checks. | Promotion, hold, abort, rollback, or fix-forward decision. |
| 8. Respond and remediate | Alert correlation, incident command, evidence collection, hypotheses, runbook, proposed fix, approval. | Verified PR or documented safe operational action. |
| 9. Learn and improve | Postmortem, outcome classification, eval case, flaky/failure cluster, policy/runbook/template update. | Reusable organizational knowledge and regression test. |

### Current performance language

ShipSense uses DORA's current five metrics: change lead time, deployment frequency, failed deployment recovery time, change fail rate, and deployment rework rate. [S03] The product must disclose calculation choices and source coverage; it must not mix incompatible definitions across teams.

### Cross-stage product invariants

- A release candidate always references an immutable artifact identity; environment promotion does not silently rebuild.
- Every delivery object retains provider link, service, environment, time, source freshness, and current/previous attempt or revision.
- Every production gate lists checks, policy, approvers, missing evidence, and what would invalidate the decision.
- Every incident can open the exact changes, rollout, config, runtime, telemetry, security, and dependency context for its time window.
- Every remediation proposal can be traced back to a reviewed hypothesis and forward to verification, PR, deployment, health, and learning outcome.

## 5. Capability map

The full product is organized into 16 connected modules. Each module remains valuable on its own and becomes stronger through shared scope, identity, history, and evidence.

| Capability | Product scope | Experience plane |
|---|---|---|
| Identity, tenancy, and policy | Organizations, teams, service accounts, SSO/SCIM, RBAC/ABAC, environment rules, approval matrices, immutable audit. | Control plane |
| Connector fabric | Signed webhooks, polling/backfill, customer collector, health, schema mapping, rate limits, replay, least-privilege scopes. | Evidence plane |
| Software catalog and temporal graph | Services, owners, repositories, pipelines, artifacts, dependencies, environments, workloads, SLOs, runbooks, and historical topology. | Knowledge plane |
| CI orchestration visibility | Pipeline DAG, jobs, tests, quality gates, logs, retry controls, queue/runner state, flaky-test detection, cache and artifact lineage. | Delivery plane |
| Supply-chain security | SAST, SCA, secrets, container/IaC/license findings, SBOM, signing, provenance, attestations, exceptions, and policy gates. | Security plane |
| Release and environment management | Release candidates, promotion, approvals, change windows, freeze periods, config comparison, release notes, multi-service sequencing. | Delivery plane |
| GitOps and progressive delivery | Desired/actual state, drift, sync health, waves/hooks, rolling, canary, blue/green, analysis gates, abort, rollback, and fix-forward. | Runtime plane |
| Kubernetes intelligence | Clusters, namespaces, workloads, pods, controllers, events, probes, resource pressure, network dependencies, and blast radius. | Runtime plane |

| Capability | Product scope | Experience plane |
|---|---|---|
| Unified observability and SLOs | OpenTelemetry-aligned logs, metrics, traces, profiles where available, dashboards, alert correlation, burn-rate views, and change overlays. | Reliability plane |
| Incident command | Alert deduplication, severity, timeline, roles, evidence, hypotheses, communications, status, postmortem, and runbook linkage. | Reliability plane |
| AI investigation | Scoped collection, multi-hypothesis reasoning, supporting/contradicting evidence, confidence caps, source coverage, budget, and abstention. | Decision plane |
| AI operations copilot | Application-scoped conversation with dynamic source planning across live telemetry, deployed code, delivery, runtime, incidents, policies, and organizational knowledge. | Decision plane |
| Governed remediation | Action proposals, policy decisions, draft branches/PRs, minimal patching, sandbox verification, existing CI, approvals, merge and GitOps observation. | Effect plane |
| Replay and evaluation lab | Frozen cases, expected outcomes, tool/prompt/model versions, deterministic fixtures, regression metrics, red-team cases, and release gates. | Learning plane |
| Engineering intelligence | Current DORA metrics, SLO and incident trends, delivery friction, rework, flaky tests, cost, adoption, and automation precision. [S03] | Analytics plane |
| Administration and operations | Plans, usage, quotas, connector diagnostics, retention, export/deletion, regional settings, feature flags, support bundles, and system health. | Management plane |

## What is shared across modules

| Shared concept | Product behavior |
|---|---|
| Scope bar | Organization, team, service, environment, region, and time follow users across compatible views. |
| AI context capsule | Every copilot turn shows the application, environment, time window, incident/release, effective role, connected-source coverage, and any approved expansion. |
| Global search | Services, repositories, runs, releases, artifacts, environments, workloads, SLOs, incidents, proposals, policies, and audit events. |
| Source provenance | Provider, object link, observed/valid time, last sync, freshness, and access limitations. |
| Unified timeline | Code, build, artifact, release, config, flag, GitOps, runtime, alert, incident, proposal, approval, and health events. |
| Evidence drawer | Fact, inference, unknown, contradiction, hash/source/time, permissions, and related claims. |
| Action framework | Eligibility, policy, approval, progress, cancellation, provider receipt, result, and observed outcome. |
| Attention queue | Risk items from delivery, reliability, security, drift, approvals, connector health, and product governance. |
| Audit correlation | From summary/score to source, decision, actor, effect, and outcome. |

# 6. Information architecture and global experience

The UI is a dense operations workspace with progressive disclosure. Navigation follows user questions, not vendor boundaries.

## Primary navigation

| Navigation area | Destinations | Primary question |
|---|---|---|
| Fleet | Overview, attention, saved views | What needs attention across the organization? |
| Catalog | Services, teams, scorecards, dependency map | What exists, who owns it, and how is it connected? |
| Delivery | Pipelines, releases, environments | What is moving through delivery, and is it safe to promote? |
| Runtime | Clusters, workloads, GitOps, drift | What is actually running, and does it match desired state? |
| Reliability | SLOs, alerts, incidents, runbooks | What user promise is at risk, and how are we responding? |
| Copilot | Global drawer, full workspace, saved conversations, watches | What can ShipSense explain about this authorized application right now? |
| Automation | Investigations, proposals, approvals, outcomes | What is ShipSense recommending, and what proof/gates apply? |
| Security | Findings, artifacts, SBOM, provenance, exceptions | Is this exact artifact acceptable for this environment? |
| Evaluate | Cases, runs, comparisons, red-team, gates | Did the product change improve safe, grounded behavior? |
| Analytics | Delivery, reliability, quality, automation, cost | Are outcomes improving, and what explains the trend? |
| Manage | Integrations, policy, audit, organization | How is ShipSense connected and governed? |

## Global app shell

- Persistent left navigation with tenant/environment identity, collapsed mode, keyboard landmarks, and unread/critical counters that do not depend on color.
- Top scope bar with breadcrumbs, environment/time controls, global search, command palette, notifications, help, and user/role context.
- AI Operations Copilot opens as a page-aware drawer or full workspace; it proposes a context capsule from the current surface and requires confirmation before meaningful scope expansion.

- Context rail for actions, source freshness, permissions, linked incident/release, and page-specific help; collapsible to preserve working space.
- Deep links retain scope and point to a stable object/revision/attempt when available.
- Every live view exposes last update, connection quality, and whether the page contains partial, historical, or access-limited data.

## Visual system

| Element | Specification |
|---|---|
| Color | Navy/blue for structure and navigation; green verified healthy/pass; gold warning/uncertain; red critical/block; neutral gray unknown/stale. Always pair with text/icon. |
| Typography | Readable sans-serif; tabular numbers for timestamps/metrics; monospace only for hashes, paths, diffs, log excerpts, and identifiers. |
| Density | Compact tables and timelines with comfortable detail panels; user-selectable compact/comfortable row density. |
| Status vocabulary | Unknown, queued, running, passed, warned, blocked, failed, canceled, stale, partial, superseded, approved, expired, verified. |
| Evidence | Source badges, observed/valid time, hash or provider ID, access label, supporting/contradicting relationship, open-in-source action. |
| Risk/confidence | Numeric score plus label, contributors, coverage, freshness, uncertainty and trend; never color-only or unexplained. |
| Motion | Subtle state transitions; animated/live updates pause under reduced-motion preference; no flashing severity indicators. |
| Accessibility | WCAG 2.2 AA target, visible focus, keyboard completion, screen-reader names, 200% zoom/reflow, accessible charts/tables, non-color meaning. |

## Reusable product components

| Component | Required behaviors |
|---|---|
| Scope bar | Persistent scope, clear reset, shareable URL, permission-aware options, historical-mode banner. |
| Status chip | Icon, text, severity, tooltip/definition, last updated; supports unknown/stale/partial. |
| Evidence card | Claim relation, source, timestamp, snippet/summary, hash/reference, open source, add/remove feedback. |
| Timeline | Zoom/filter, change types, facts/inference, correlation, historical gaps, export. |
| Pipeline DAG | Keyboard navigation, zoom, attempt/state, critical path, failure focus, accessible list alternative. |

| Component | Required behaviors |
|---|---|
| Dependency graph | Declared/observed edges, confidence/source, time slider, blast-radius mode, accessible table alternative. |
| Diff viewer | File/path scope, syntax, added/removed summary, test weakening alerts, comments, accessible unified mode. |
| Gate list | Requirement, source, result, freshness, policy/approver, retry or remediation, material-change warning. |
| Approval panel | Exact object/version, risk, checks, eligible approvers, separation, expiry, decision rationale. |
| AI context capsule | Application, environment, time, incident/release, effective role, source coverage, memory state, visible reset and scope-expansion control. |
| AI source plan | Ordered sources, reason, permission/freshness/cost, progress, cancel, optional expansion, and resulting evidence links. |
| Command palette | Search/navigation first; action suggestions filtered by permissions and confirmation level. |
| Activity/audit drawer | Actor, action, object/version, before/after, decision, correlation, export. |
| Provider handoff | Open exact provider object in a new tab with a clear external-boundary label. |

## Global state behavior

| State | Required experience |
|---|---|
| Loading | Skeleton preserves page structure; no fake green state; action buttons disabled until authority and object version resolve. |
| Empty | Explain why empty, what good state looks like, and a permission-appropriate setup or navigation action. |
| No results | Preserve filters; show active constraints; offer clear-all; never imply no underlying data exists. |
| Partial coverage | Show which integrations, services, time ranges, or evidence types are missing and how this limits conclusions. |
| Stale | Display source, last successful sync, expected freshness, affected insights/actions, and retry/open-in-provider options. |
| Disconnected | Keep historical state read-only; disable current controls; explain token/scope/reconnect owner and impact. |
| Rate-limited | Show provider, reset estimate, queued work, degraded features, and whether manual retry would worsen the condition. |
| Permission denied | Do not reveal protected object contents; explain required role/scope and provide access-request route. |
| Policy blocked | Show decision, rule/version, matching input, obligation, override route, and immutable correlation ID. |

| State | Required experience |
|---|---|
| Approval required | Show approver role, eligible people/teams, expiry, separation-of-duty rule, and what change invalidates approval. |
| Action in progress | Show durable steps, current stage, elapsed time, cancel semantics, provider receipt, and safe reload behavior. |
| Canceled | Distinguish user cancel, superseded work, timeout, provider cancel, and policy revocation; preserve partial evidence. |
| Failed | Show user-safe summary, affected scope, retry safety, correlation ID, diagnostics path, and fallback workflow. |
| Conflict | Show competing edits/approvals/state versions; require refresh/compare; never silently last-write-wins for material decisions. |
| Unknown | Use neutral style and explicit reason; never map absence of evidence to healthy, passed, or safe. |
| Deleted/expired | Preserve tombstone, identity, reason, retention boundary, and related audit links without exposing removed content. |

## 7. Detailed feature specifications

Each feature is defined by a user promise, included experience, available actions, behavioral rules, and release scope. All 17 features operate within a unified Project context—connecting GitHub version control, Jenkins/GitHub CI pipelines, Vercel/Netlify/Railway/Render deployments, Grafana/OpenTelemetry observability, Slack alerts, and Jira project management so AI can assist across the entire CI/CD lifecycle while preserving existing architectural and governance boundaries.

### FEAT-01 — Fleet Command Center

| | |
|---|---|
| **Product promise** | Unify health, risk, and status across your entire project workspace by orchestrating signals from GitHub, Jenkins, Vercel, Netlify, Railway, Render, Grafana, OpenTelemetry, Slack, and Jira. |
| **Included experience** | Project-wide health summary; active Jenkins/GitHub builds; cloud deployment statuses (Vercel, Netlify, Railway, Render); OpenTelemetry/Grafana SLO burn; Jira issue links; Slack alert streams; attention queue; saved views. |
| **Primary user actions** | Filter by team/tier/environment/region; acknowledge attention item; open incident; compare time windows; export reviewed summary; save dashboard. |
| **Behavioral rules** | Risk must show contributing signals and freshness. Unknown is not healthy. Leadership trends never hide the underlying service/run/incident evidence. |
| **Delivery scope** | Release 1 core; portfolio and cost extensions in Release 2. |

## FEAT-02 — Software Catalog and Knowledge Graph

| | |
|---|---|
| **Product promise** | Maintain an AI-assisted, project-centric knowledge graph mapping GitHub repositories, Jenkins/GitHub Actions pipelines, Vercel/Netlify/Railway/Render deployments, Jira tasks, and OpenTelemetry/Grafana telemetry. |
| **Included experience** | Project directory; ownership; GitHub repo mappings; Jenkins/GitHub Actions pipeline DAGs; deployment targets (Vercel, Netlify, Railway, Render); Grafana/OTel SLO mappings; Jira issue links; catalog health scorecards. |
| **Primary user actions** | Search; filter; claim ownership; resolve ambiguous match; edit metadata; attach runbook/SLO; inspect historical topology; view blast radius. |
| **Behavioral rules** | Every relationship must identify source and time. Ambiguous identities require review. Human override is visible and reversible. |
| **Delivery scope** | Release 1 core; extensible scorecards and custom entity types in Release 2. |

## FEAT-03 — Service 360

| | |
|---|---|
| **Product promise** | Provide a unified 360-degree operational cockpit for every project service, linking code, builds, cloud deployments, telemetry, and Jira tasks. |
| **Included experience** | Service overview; GitHub repo link; Jenkins/Actions builds; Vercel/Netlify/Railway/Render deployment status; Grafana/OTel dashboards & SLOs; Slack notification hooks; Jira issue tracking; environment parity comparisons. |
| **Primary user actions** | Manually trigger builds/deployments or instruct AI to manage service workflows; compare environments; manage Jira tasks; inspect recent changes; configure Slack alerts; request remediation. |
| **Behavioral rules** | Tabs share the same service/time/environment scope. Partial integrations are labeled. Sensitive values are never displayed. |
| **Delivery scope** | Release 1 core. |

## FEAT-04 — Pipeline and Test Intelligence

| | |
|---|---|
| **Product promise** | Transform provider-specific CI runs across Jenkins and GitHub Actions into a single, project-centric pipeline intelligence view. |
| **Included experience** | Jenkins/GitHub Actions DAGs; job attempts; test suite normalization; failure clustering; flaky test detection; Slack alert triggers; Jira task creation for build breakages. |
| **Primary user actions** | Manually trigger, cancel, retry, or parameterize Jenkins and GitHub Actions runs, or delegate pipeline management to AI; compare runs; inspect logs; assign failures; create/sync Jira issues. |

| | |
|---|---|
| **Behavioral rules** | Rerun never erases first failure. Superseded commit is clear. Infrastructure and product failures are distinguished. Controls are policy- and provider-aware. |
| **Delivery scope** | GitHub Actions in Release 1; provider catalog expands in Release 2. |

## FEAT-05 — Supply-Chain Assurance

| | |
|---|---|
| **Product promise** | Ensure every GitHub commit and build artifact is policy-compliant, signed, and safe for cloud deployment. |
| **Included experience** | SAST/SCA/secrets findings; SBOM verification; artifact digest checks; GitHub commit signing; Slack alert notifications; Jira security task sync; cloud deployment gating. |
| **Primary user actions** | Inspect finding; compare scans; verify artifact; request exception; approve/reject exception; open remediation proposal; block/unblock release through policy. |
| **Behavioral rules** | Tags do not establish identity; digest does. Exceptions are scoped, time-bound, justified, and re-evaluated on expiry. ShipSense describes posture, not certification. |
| **Delivery scope** | Essential artifact evidence in Release 1; full security center in Release 2. |

## FEAT-06 — Release Command Center

| | |
|---|---|
| **Product promise** | Coordinate release promotion across cloud providers (Vercel, Netlify, Railway, Render) backed by verified Jenkins/GitHub CI runs and Jira tracking. |
| **Included experience** | Release candidate lineage; Vercel/Netlify/Railway/Render environment progression; Jira release linkage; Slack approval alerts; OpenTelemetry rollout verification; automated rollback triggers. |
| **Primary user actions** | Manually trigger, promote, hold, resume, or rollback deployments across Vercel, Netlify, Railway, and Render, or use AI to orchestrate release promotions and Jira release gates. |
| **Behavioral rules** | Promotion uses the same artifact digest. Approval binds to a specific release and expires on material change. Merge/deploy is not success until health verification passes. |
| **Delivery scope** | Release 1 core; multi-service trains in Release 2. |

## FEAT-07 — Environment and Configuration Intelligence

| | |
|---|---|
| **Product promise** | Detect configuration drift and environment discrepancies across Vercel, Netlify, Railway, Render, and GitHub environments. |
| **Included experience** | Side-by-side environment comparisons; Vercel/Netlify/Railway/Render config drift detection; GitHub environment secret references; Slack notifications for unapproved drift. |

| | |
|---|---|
| **Primary user actions** | Compare environments; manually sync or update environment configuration metadata/variables across Vercel, Netlify, Railway, Render, and GitHub, or use AI to reconcile configuration drift. |
| **Behavioral rules** | Never reveal secret values. Expected differences can be documented; ignored differences require owner and expiry. |
| **Delivery scope** | Release 1 core. |

## FEAT-08 — GitOps and Progressive Delivery

| | |
|---|---|
| **Product promise** | Expose desired state versus live deployment state across GitHub PRs, cloud targets (Vercel, Netlify, Railway, Render), and OpenTelemetry metrics. |
| **Included experience** | GitHub manifest synchronization; cloud deployment status; OpenTelemetry canary analysis; Slack progressive rollout alerts; automated abort/rollback triggers. |
| **Primary user actions** | Manually trigger deployments, adjust traffic weights, promote, pause, or rollback across Vercel, Netlify, Railway, and Render, or let AI manage progressive rollouts under policy. |
| **Behavioral rules** | Git and connected deployment providers remain synchronized. Both manual operator controls and AI-driven deployment actions are supported with audit and policy checks. |
| **Delivery scope** | Argo CD/Rollouts in Release 1; more providers in Release 2. |

## FEAT-09 — Kubernetes Runtime Explorer

| | |
|---|---|
| **Product promise** | Explain cluster and workload health alongside Vercel/Netlify/Railway/Render deployment states and OpenTelemetry signals. |
| **Included experience** | Clusters; workloads; pod health; OpenTelemetry metrics overlay; GitHub commit alignment; Jira issue creation for failing workloads. |
| **Primary user actions** | View live runtime telemetry and logs; manually restart instances/services, redeploy, or scale workloads across Vercel, Netlify, Railway, Render (and clusters), or instruct AI to execute runtime recovery actions. |
| **Behavioral rules** | Controller and cloud platform semantics are respected. Both manual runtime controls and governed AI operational actions are available for authorized roles. |
| **Delivery scope** | Release 1 core; cross-cloud inventory in Release 3. |

## FEAT-10 — Observability and SLO Center

| | |
|---|---|
| **Product promise** | Connect OpenTelemetry and Grafana signals directly to project releases, GitHub commits, and cloud deployment targets. |
| **Included experience** | Grafana dashboard embeds; OpenTelemetry trace/metric links; deployment change overlays (Vercel, Netlify, Railway, Render); SLO burn rate tracking; Slack alert triggers. |
| **Primary user actions** | Change scope/time; open trace/log/metric; compare baseline; create SLO; attach evidence; start incident; export reviewed snapshot. |
| **Behavioral rules** | ShipSense indicates partial/missing telemetry. The product does not silently equate correlation with causation. |
| **Delivery scope** | Prometheus/Grafana/OpenTelemetry in Release 1; provider expansion in Release 2. |

## FEAT-11 — Incident Command Workspace

| | |
|---|---|
| **Product promise** | Centralize incident response by linking Slack alerts, OpenTelemetry/Grafana telemetry, GitHub changes, and Jira tickets into a single command space. |
| **Included experience** | Slack alert ingestion; incident timelines; OpenTelemetry/Grafana evidence links; Jira issue syncing; AI-assisted remediation PR generation. |
| **Primary user actions** | Acknowledge and manage alerts; manually or via AI create/transition Jira incident tickets, dispatch Slack updates, trigger diagnostic/recovery pipelines, and execute runbook actions. |
| **Behavioral rules** | Facts, inference, and unknowns are visually separate. Split/merge keeps provenance. Resolution requires observed recovery and reviewer confirmation. |
| **Delivery scope** | Release 1 signature experience. |

## FEAT-12 — AI Evidence Investigator

| | |
|---|---|
| **Product promise** | Produce inspectable, AI-driven root-cause investigations by analyzing evidence across GitHub, Jenkins, Vercel, Netlify, Railway, Render, Grafana, OpenTelemetry, Slack, and Jira. |
| **Included experience** | Cross-tool evidence collection (GitHub commits, Jenkins logs, Grafana metrics, Slack chats, Jira issues); hypothesis ranking; contradiction tracking; confidence capping. |
| **Primary user actions** | Start/cancel; adjust authorized scope; inspect source; mark evidence relevant/incorrect; challenge hypothesis; request missing evidence; accept or reject finding. |
| **Behavioral rules** | Retrieved content is data, not instruction. Confidence is capped by coverage. The investigator must abstain when evidence, permission, or tool reliability is insufficient. |
| **Delivery scope** | Release 1 signature experience. |

## FEAT-13 — Governed Remediation Proposals

| | |
|---|---|
| **Product promise** | Generate verified, governed remediation PRs in GitHub, linking fixes to Jira issues and verifying via Jenkins/GitHub Actions before cloud deployment. |
| **Included experience** | GitHub PR drafting; Jenkins/Actions pre-verification; Jira issue updating; Slack review notifications; cloud target impact preview (Vercel, Netlify, Railway, Render). |
| **Primary user actions** | Generate, edit, approve, and execute code/config fixes, CI adjustments, deployment rollbacks, and Jira task updates manually or via AI. |
| **Behavioral rules** | Proposal is not execution. ShipSense cannot approve its own proposal. No direct production write in Release 1. Test weakening and unrelated changes are blocked. |
| **Delivery scope** | Release 1 signature experience. |

## FEAT-14 — Replay and Evaluation Lab

| | |
|---|---|
| **Product promise** | Evaluate AI agent performance against frozen project incidents, CI pipeline runs, and cloud deployment failure scenarios. |
| **Included experience** | Case library with historical GitHub PRs, Jenkins logs, and OpenTelemetry traces; regression testing; model/prompt comparisons; evaluation dashboards. |
| **Primary user actions** | Create case; run replay; compare versions; inspect miss; label outcome; promote baseline; export report; block/approve product change. |
| **Behavioral rules** | Cases are tenant-isolated or synthetic. Expected labels are hidden during execution. A passing average cannot hide an unsafe individual case. |
| **Delivery scope** | Release 1 signature differentiator; broader live benchmarks in Release 2. |

## FEAT-15 — Engineering Intelligence

| | |
|---|---|
| **Product promise** | Analyze engineering velocity, stability, and delivery metrics aggregated across GitHub, Jenkins, Vercel, Netlify, Railway, Render, and Jira. |
| **Included experience** | DORA metrics calculation; lead time analysis from Jira ticket to deployment; change fail rates across cloud targets; Jenkins/GitHub pipeline efficiency trends. |
| **Primary user actions** | Choose definition/window; compare teams with context; drill down; annotate change; export reviewed report; set objective. |
| **Behavioral rules** | Definitions and source lineage are visible. Metrics are not used to rank individuals. Missing data and organizational changes are disclosed. |

| | |
|---|---|
| **Delivery scope** | Core DORA/reliability in Release 1; advanced cost and custom analytics in Release 2. |

### FEAT-16 — Integration and Administration Center

| | |
|---|---|
| **Product promise** | Manage and govern core project integrations including GitHub, Jenkins, Vercel, Netlify, Railway, Render, Grafana, OpenTelemetry, Slack, and Jira. |
| **Included experience** | Integration management catalog; OAuth/API key setup for core tools; webhook health monitoring; permission scoping; sync backfill controls. |
| **Primary user actions** | Install/revoke connector; remap scope; retry backfill; manage roles/policy; export audit; request deletion; use break-glass; disable AI tool/effect. |
| **Behavioral rules** | High-risk changes require reauthentication and audit. Removing a connector previews affected features and preserves historical evidence per retention. |
| **Delivery scope** | Release 1 essentials; enterprise identity/residency in Release 3. |

### FEAT-17 — AI Operations Copilot

| | |
|---|---|
| **Product promise** | Act as an active AI operational controller and assistant across GitHub, Jenkins, Vercel, Netlify, Railway, Render, Grafana, OpenTelemetry, Slack, and Jira—capable of both answering cited project questions and executing authorized CI/CD, deployment, alerting, and Jira management actions. |
| **Included experience** | Interactive chat drawer; context capsule; multi-source query planner across core tools (GitHub, Jenkins, Vercel, Netlify, Railway, Render, Grafana, OTel, Slack, Jira); cited evidence answers; governed remediation handoffs. |
| **Primary user actions** | Ask project questions; command AI to trigger Jenkins/GitHub builds, manage Vercel/Netlify/Railway/Render deployments, create/update Jira issues, send Slack notifications, or draft remediation PRs; approve and execute governed action plans. |
| **Behavioral rules** | Effective user permissions are resolved before every retrieval or action. Read-only queries execute immediately with citations; operational control commands execute through the governed action framework with user confirmation/approval and full auditability. |
| **Delivery scope** | Release 1 for read, explain, compare and investigation handoff; proactive and drafting extensions in Release 2. |

## 8. Screen-by-screen UI specification

These screen blueprints define the core information, actions, states, and navigation required for a coherent full product. The mockups are directional layouts, not final visual design.

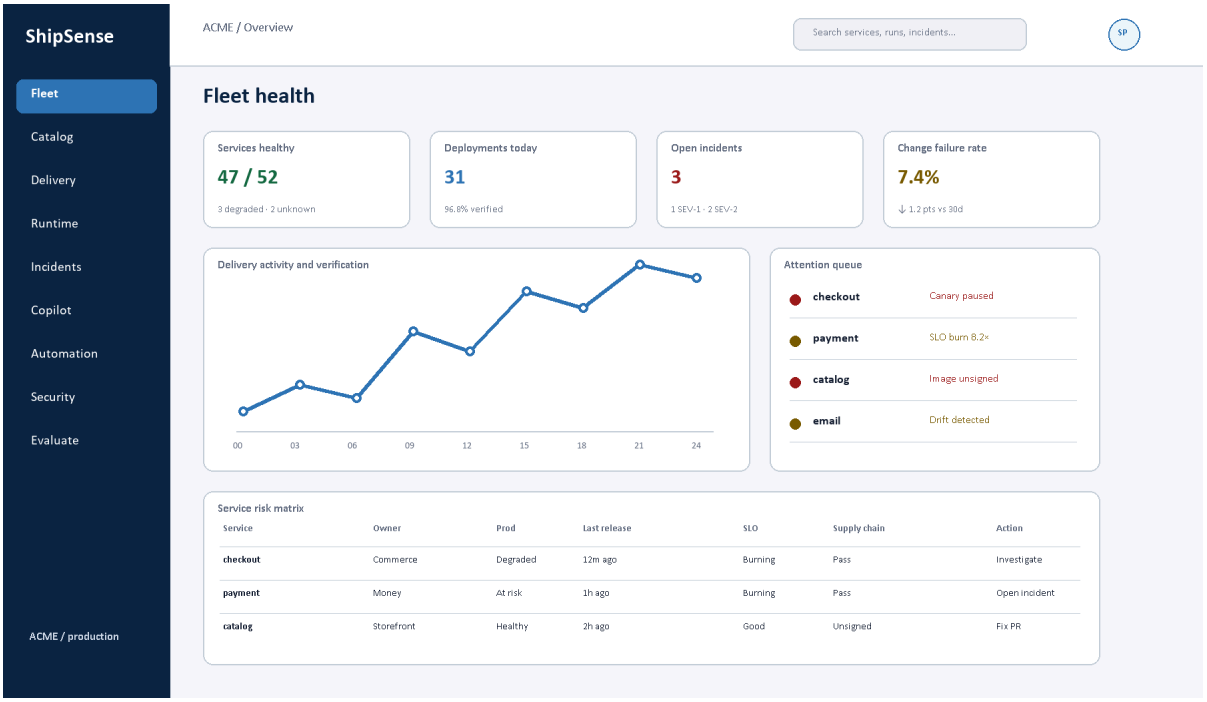



*Figure 2. Fleet health directional UI.*

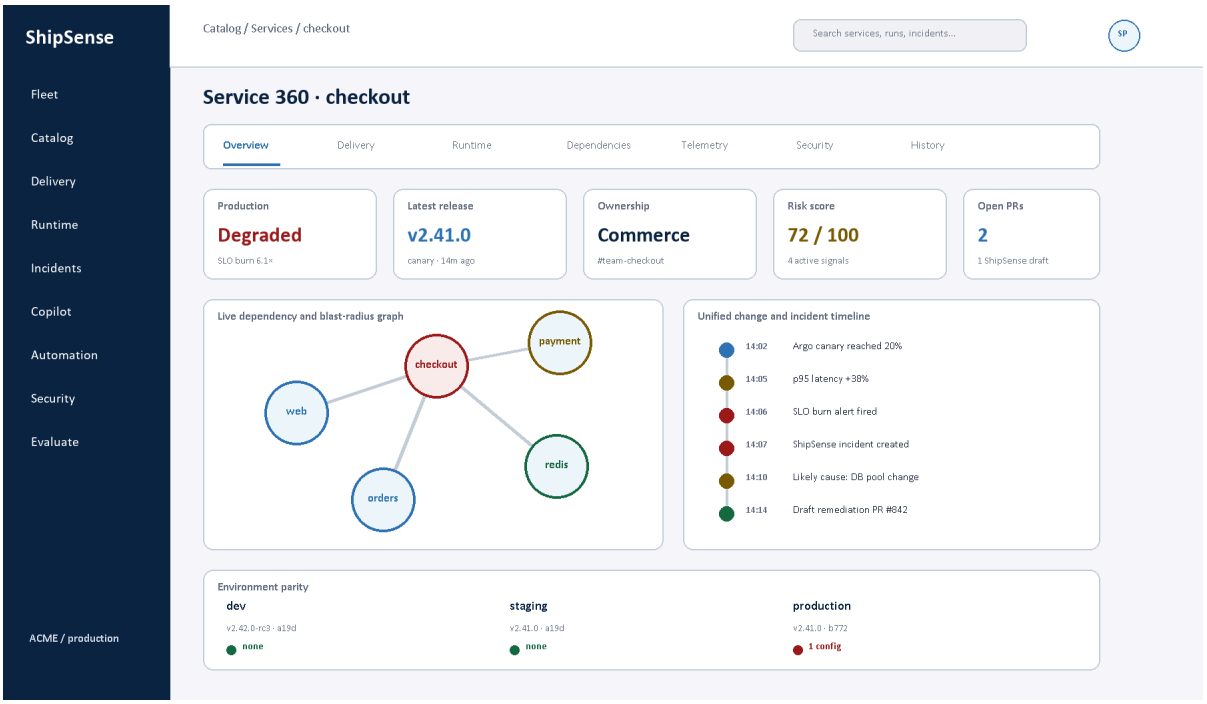



*Figure 3. Service 360 directional UI.*

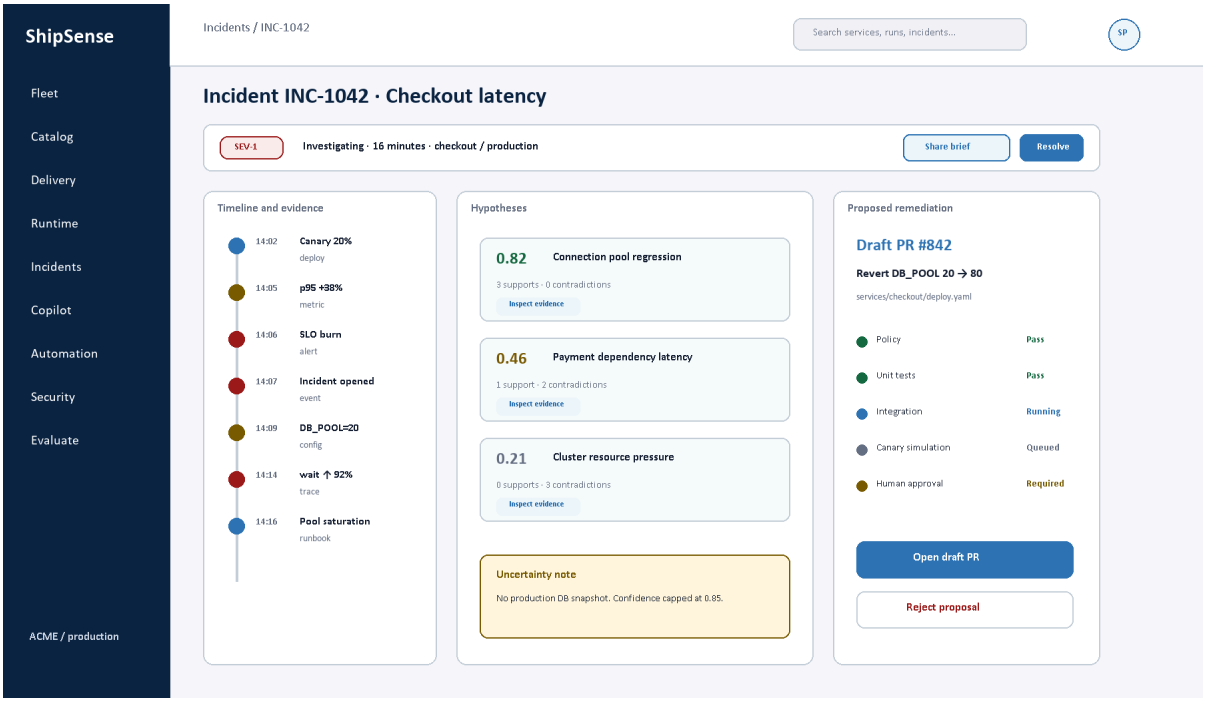



*Figure 4. Incident workspace directional UI.*

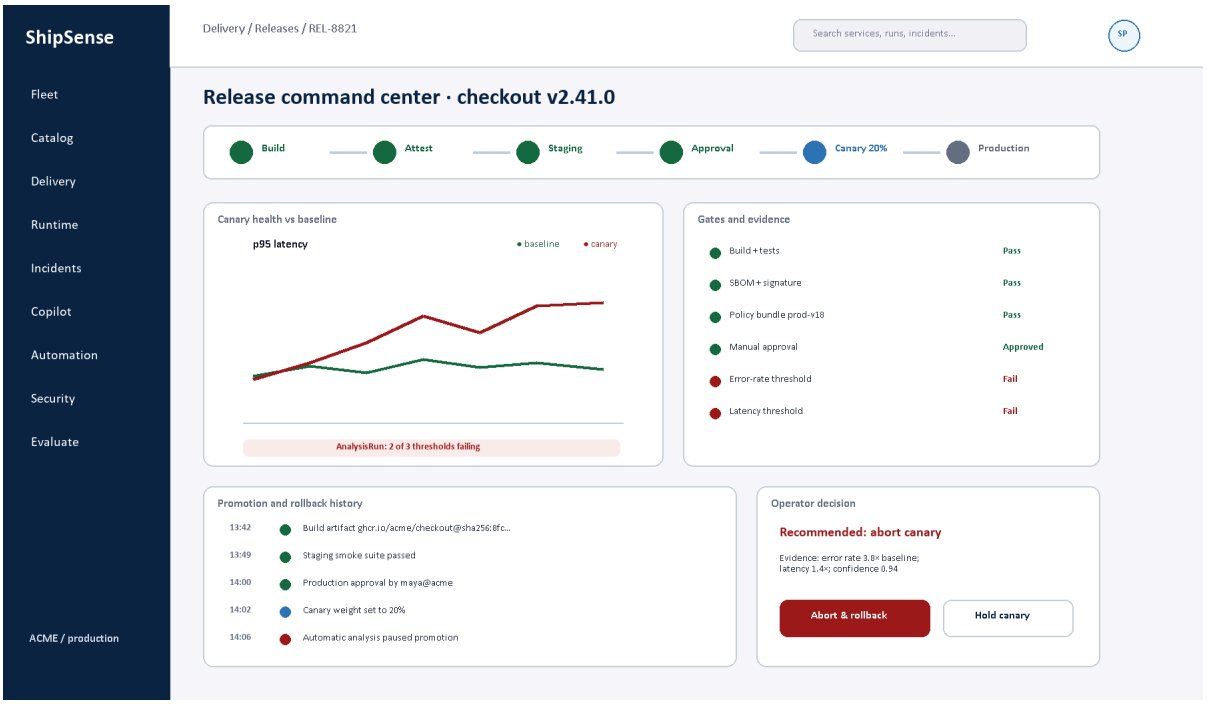



*Figure 5. Release command center directional UI.*

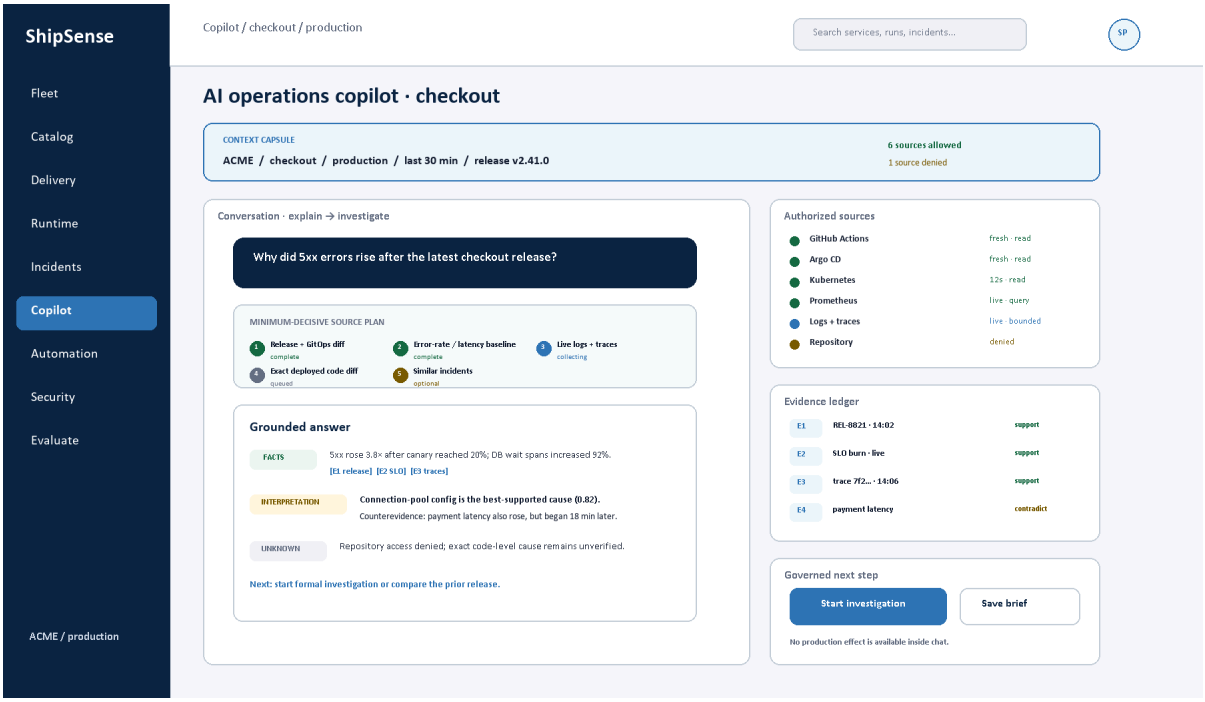



*Figure 6. AI Operations Copilot directional UI.*

## Route inventory

| Route | Screen | Primary user question |
|---|---|---|
| /overview | Fleet health | What needs attention now, what changed recently, and which service or release poses the greatest risk? |
| /catalog/services | Software catalog | What software exists, who owns it, where does it run, and how complete is its operational context? |
| /catalog/services/:id | Service 360 | What is this service, what changed, is it healthy, and what should I do next? |
| /delivery/pipelines | Pipeline explorer and run detail | Where is delivery blocked, why did this attempt fail, and is the failure new, flaky, or infrastructural? |
| /delivery/releases | Release command center | Can this exact candidate move forward safely, and what should happen if verification degrades? |
| /delivery/environments | Environment comparison | What differs between two environments, and which differences are risky, expected, or unexplained? |
| /runtime/clusters | Cluster and workload explorer | Which runtime object is unhealthy, which controller owns it, and what changed around the failure? |
| /runtime/gitops | GitOps applications | Does actual runtime match versioned desired state, and where is reconciliation stuck? |
| /reliability/slos | SLO center | Which user promise is at risk, how quickly is budget burning, and what changes correlate with it? |
| /incidents | Incident queue | Which incidents are active, who owns them, what is their impact, and where is response stalled? |

| Route | Screen | Primary user question |
|---|---|---|
| /incidents/:id | Incident workspace | What happened, what do we know, what remains uncertain, and which safe action should responders take? |
| /assistant | AI operations copilot | What is happening in this authorized application, what evidence supports the answer, and which governed next step is appropriate? |
| /automation/proposals | Action proposal and remediation review | Why is this change proposed, what exactly will it affect, how was it verified, and who must approve it? |
| /evaluate/cases | Replay and evaluation lab | Did the latest product/model/tool/policy version improve reliable behavior without adding unsafe failures or excessive cost? |
| /security | Supply-chain posture | Which release or service is exposed, which artifact is affected, and does policy allow promotion? |
| /analytics | Engineering intelligence | How are speed, stability, reliability, quality, and automation outcomes changing—and why? |
| /integrations | Integration catalog and health | Which tools are connected, what data/actions are authorized, and is the connection complete and healthy? |
| /policies | Policy and approval studio | Which actions/releases are allowed, denied, or require approval, and why? |
| /audit | Audit explorer | Who or what accessed data, made a decision, proposed an action, approved it, and observed the outcome? |
| /admin | Organization administration | How is ShipSense configured, governed, limited, supported, exported, or retired for this organization? |

## Fleet health — /overview

| | |
|---|---|
| **User question** | What needs attention now, what changed recently, and which service or release poses the greatest risk? |
| **Primary regions** | Global scope bar; four outcome cards; delivery/incident time series; attention queue; service risk table; connector freshness drawer; saved views. |
| **Actions and navigation** | Change organization/team/environment/time; sort and filter risk; acknowledge item; create incident; open Service 360/release/pipeline; save/share view. |
| **Required states** | First-use setup; no active risk; partial integration coverage; stale source; permission-limited totals; surge mode during major incident; export-ready reviewed snapshot. |
| **Interaction rules** | Every score opens its contributing signals. Unknown and stale never render as green. Default sort favors severity, customer impact, active change, and freshness. |

## Software catalog — /catalog/services

| | |
|---|---|
| **User question** | What software exists, who owns it, where does it run, and how complete is its operational context? |
| **Primary regions** | Search/query builder; filters; catalog health summary; table/card toggle; bulk-action tray; unresolved mappings; import/setup guidance. |

| | |
|---|---|
| **Actions and navigation** | Search; filter by owner/tier/lifecycle/tag/environment; claim service; bulk assign; open service; resolve mapping; attach docs/runbook; export. |
| **Required states** | No services connected; discovery running; duplicate candidate; orphan runtime; missing owner/SLO/runbook; archived service; access-restricted service. |
| **Interaction rules** | Search results explain matches. Bulk edits require preview. Source and last-reconciled time appear on every externally derived field. |

## Service 360 — /catalog/services/:id

| | |
|---|---|
| **User question** | What is this service, what changed, is it healthy, and what should I do next? |
| **Primary regions** | Persistent service header; health/owner/release/risk cards; tab bar; dependency graph; unified activity timeline; environment parity strip; contextual action rail. |
| **Actions and navigation** | Start investigation; open change/release/workload/evidence; compare environments; subscribe; edit ownership; add runbook/SLO; request remediation. |
| **Required states** | Healthy; degraded; unknown; no runtime; multiple deployables; ownership conflict; telemetry partial; historical time-travel; incident context pinned. |
| **Interaction rules** | Environment and time scope persist across tabs. Related objects open without losing scope. AI summaries always link to facts and uncertainty. |

## Pipeline explorer and run detail — /delivery/pipelines

| | |
|---|---|
| **User question** | Where is delivery blocked, why did this attempt fail, and is the failure new, flaky, or infrastructural? |
| **Primary regions** | Pipeline list; run filters; DAG; job inspector; attempt selector; log viewer; test/failure tabs; artifacts/checks; provider link; action bar. |
| **Actions and navigation** | Cancel/retry if allowed; compare attempts/runs; search logs; pin line as evidence; assign failure; create issue/incident; open artifact/release. |
| **Required states** | Queued; running; passed; failed; infra failed; canceled; timed out; skipped; neutral; superseded; logs expired; provider unavailable; fork PR restricted. |
| **Interaction rules** | Current commit and attempt are prominent. A retry is a new attempt. Log secrets are redacted; redaction is disclosed. Controls state why unavailable. |

## Release command center — /delivery/releases

| | |
|---|---|
| **User question** | Can this exact candidate move forward safely, and what should happen if verification degrades? |
| **Primary regions** | Candidate header; promotion stepper; artifact lineage; environment matrix; approvals; gate/evidence list; progressive rollout chart; timeline; operator decision panel. |

| | |
|---|---|
| **Actions and navigation** | Request/approve/deny; hold/resume/abort; open diff; compare baseline; publish update; invoke reviewed rollback; create incident. |
| **Required states** | Draft; waiting checks; waiting approval; scheduled; frozen; deploying; paused; analyzing; promoted; aborted; rolled back; partially deployed; superseded. |
| **Interaction rules** | Approval is bound to candidate and policy version. Material change invalidates it. Health decision shows thresholds, source freshness, and missing signals. |

## Environment comparison — /delivery/environments

| | |
|---|---|
| **User question** | What differs between two environments, and which differences are risky, expected, or unexplained? |
| **Primary regions** | Environment selectors; artifact/config/schema/flag/policy/dependency comparison; difference filter; expected-difference registry; source panel. |
| **Actions and navigation** | Choose baseline/target; pin difference; mark expected with owner/expiry; open source PR/config; attach to release/incident; create parity issue. |
| **Required states** | In sync; expected variance; unexplained drift; source stale; environment inaccessible; secret metadata hidden; no common service revision. |
| **Interaction rules** | Secret values never render. Ignored differences expire. A difference is not labeled causal unless evidence connects it to observed behavior. |

## Cluster and workload explorer — /runtime/clusters

| | |
|---|---|
| **User question** | Which runtime object is unhealthy, which controller owns it, and what changed around the failure? |
| **Primary regions** | Cluster/namespace tree; inventory table; controller/workload detail; replicas/probes/resources; events; rollout/revision; dependency graph; timeline. |
| **Actions and navigation** | Filter; time-travel; open related release/config; copy sanitized diagnostics; add evidence; start investigation; link to incident. |
| **Required states** | Connected; collector lag; cluster unreachable; RBAC partial; workload progressing/degraded; crash loop; probe failure; resource pressure; deleted object in historical view. |
| **Interaction rules** | Production is read-only in Release 1. Desired/current/available are distinct. Deleted historical objects remain visible only inside valid time scope. |

## GitOps applications — /runtime/gitops

| | |
|---|---|
| **User question** | Does actual runtime match versioned desired state, and where is reconciliation stuck? |
| **Primary regions** | Application list; sync/health summary; desired vs live diff; resources; phases/waves/hooks; sync history; drift; revision links; action panel. |

| | |
|---|---|
| **Actions and navigation** | Open source revision/PR; request allowed sync; hold/resume/abort rollout; record exception; create incident; compare prior sync. |
| **Required states** | Synced; out-of-sync; progressing; degraded; missing; unknown; hook failed; partial sync; prune blocked; manual drift; controller unavailable. |
| **Interaction rules** | ShipSense distinguishes observed state from desired state. Destructive sync or prune requires explicit provider permission and policy; default is guidance only. |

## SLO center — /reliability/slos

| | |
|---|---|
| **User question** | Which user promise is at risk, how quickly is budget burning, and what changes correlate with it? |
| **Primary regions** | SLO portfolio; error-budget cards; burn-rate matrix; SLI chart; release/incident overlays; ownership; alert policy; coverage gaps. |
| **Actions and navigation** | Create/edit draft SLO; choose window; compare baseline; open evidence; start incident; annotate known event; export reliability review. |
| **Required states** | Healthy; fast burn; slow burn; exhausted; no data; delayed data; objective draft; invalid query; ownership missing. |
| **Interaction rules** | Missing data is not success. Objective/query changes create versioned history. Leadership views retain definition and source context. |

## Incident queue — /incidents

| | |
|---|---|
| **User question** | Which incidents are active, who owns them, what is their impact, and where is response stalled? |
| **Primary regions** | Severity/status filters; active queue; alert groups; commander/on-call; timers; affected services; automation progress; handoff panel; recently resolved. |
| **Actions and navigation** | Acknowledge; assign; change severity; merge/split; start/stop investigation; open workspace; subscribe; bulk triage low-severity candidates. |
| **Required states** | New; acknowledged; investigating; mitigated; monitoring; resolved; reopened; duplicate; external dependency; automation blocked; no responder. |
| **Interaction rules** | Merge/split preview evidence and affected timeline. Severity suggestion never silently changes declared severity. Unowned critical incidents remain pinned. |

## Incident workspace — /incidents/:id

| | |
|---|---|
| **User question** | What happened, what do we know, what remains uncertain, and which safe action should responders take? |

| | |
|---|---|
| **Primary regions** | Incident header; role/status bar; chronological timeline; evidence drawer; hypothesis board; dependency/blast radius; communications; proposal/approval rail; postmortem tab. |
| **Actions and navigation** | Add fact/inference/unknown; attach evidence; run/cancel investigation; challenge hypothesis; execute runbook checkpoint; review proposal; publish update; resolve/reopen. |
| **Required states** | AI collecting; source denied; budget exhausted; evidence conflict; no confident cause; proposal verifying; approval expired; recovery observed; postmortem overdue. |
| **Interaction rules** | Facts, inference, unknowns, and external statements have distinct labels. Timeline is append-only with corrections. Resolution requires observed recovery and owner confirmation. |

## AI operations copilot — /assistant

| | |
|---|---|
| **User question** | What is happening in this authorized application, what evidence supports the answer, and which governed next step is appropriate? |
| **Primary regions** | Persistent context capsule; conversation and suggested questions; source-plan progress; authorized-source drawer; evidence ledger; fact/interpretation/unknown answer blocks; conversation memory; governed action rail. |
| **Actions and navigation** | Ask/refine; pin application/environment/time/incident/release; approve scope expansion; pause; inspect/correct source; compare; start investigation; save reviewed brief; draft issue/runbook/proposal; export/reset. |
| **Required states** | No application selected; ambiguous entity; ready; planning; collecting; source stale/denied/rate-limited; contradiction; budget warning/exhausted; partial answer; abstained; scope expansion requested; action handoff pending; conversation expired. |
| **Interaction rules** | The current page supplies a proposed scope, never hidden authority. Every retrieval uses effective user permission. Material claims are cited. Memory cannot cross tenant/user/application boundaries. Effects cannot execute inside chat. |

## Action proposal and remediation review — /automation/proposals

| | |
|---|---|
| **User question** | Why is this change proposed, what exactly will it affect, how was it verified, and who must approve it? |
| **Primary regions** | Proposal summary; causal chain; diff; affected paths/services; alternatives; test/security/policy results; risk/blast radius; rollout/rollback; approval history; PR status; outcome. |
| **Actions and navigation** | Request edit/reverify; reject with reason; request approval; open draft PR; compare with human change; cancel; convert outcome to eval case. |
| **Required states** | Draft; generating; verification queued/running/passed/failed; policy blocked; approval required/expired; PR open/merged/closed; rollout healthy/regressed; withdrawn. |
| **Interaction rules** | No hidden changes. Test deletions/weakening are highlighted. ShipSense never self-approves. Production result remains unknown until observed health verification. |

## Replay and evaluation lab — /evaluate/cases

| | |
|---|---|
| **User question** | Did the latest product/model/tool/policy version improve reliable behavior without adding unsafe failures or excessive cost? |
| **Primary regions** | Dataset browser; case detail; run configuration; comparison table; evidence/cause/action scores; unsafe-case panel; cost/latency; failure analysis; release gate. |
| **Actions and navigation** | Create/sanitize case; run replay; compare versions; inspect transcript/evidence; label outcome; promote baseline; exclude with reviewed reason; export benchmark. |
| **Required states** | Ready; running; canceled; nondeterministic dependency; fixture invalid; unsafe failure; regression; improvement; awaiting human label; tenant-restricted. |
| **Interaction rules** | Unsafe individual cases cannot be averaged away. Expected labels are protected during run. Every result lists exact product/model/prompt/tool/policy versions. |

## Supply-chain posture — /security

| | |
|---|---|
| **User question** | Which release or service is exposed, which artifact is affected, and does policy allow promotion? |
| **Primary regions** | Posture summary; findings by type/severity; artifact/SBOM/provenance explorer; policy decisions; exceptions; expiry queue; affected releases/services. |
| **Actions and navigation** | Inspect finding/component; verify artifact; compare scans; request/approve exception; open remediation; block release through policy; export reviewed report. |
| **Required states** | Verified; unsigned; provenance missing; policy denied; exception active/expiring/expired; scanner stale; conflicting scanners; not exploitable determination pending. |
| **Interaction rules** | Findings preserve source and scanner time. Exception does not remove finding. Artifact identity is digest-based. Product language avoids unsupported compliance claims. |

## Engineering intelligence — /analytics

| | |
|---|---|
| **User question** | How are speed, stability, reliability, quality, and automation outcomes changing—and why? |
| **Primary regions** | Metric definition bar; DORA cards; trend charts; distribution; team/service filters; annotations; friction drivers; automation outcomes; cost; drill-down table. |
| **Actions and navigation** | Change definition/window; compare cohorts; annotate known organizational change; drill to runs/releases/incidents; schedule/export reviewed report. |
| **Required states** | Insufficient history; source coverage changed; team reorganized; outlier; definition version changed; access-limited aggregation. |
| **Interaction rules** | No individual ranking. Median/distribution preferred over misleading averages. Metric definition and lineage travel with exports. |

## Integration catalog and health — /integrations

| | |
|---|---|
| **User question** | Which tools are connected, what data/actions are authorized, and is the connection complete and healthy? |
| **Primary regions** | Catalog; installed integrations; permissions; mapped scope; backfill progress; health/lag; rate-limit budget; errors; field mapping; revoke impact. |
| **Actions and navigation** | Install; test; map; backfill; pause; retry; change scope; rotate/reconnect; download diagnostics; revoke. |
| **Required states** | Not installed; authorizing; connected; backfilling; healthy; degraded; rate-limited; permission lost; schema changed; disconnected; revoked. |
| **Interaction rules** | Permission request explains feature impact. Revoke preview lists affected screens/actions and retention outcome. Historical evidence is not silently deleted. |

## Policy and approval studio — /policies

| | |
|---|---|
| **User question** | Which actions/releases are allowed, denied, or require approval, and why? |
| **Primary regions** | Policy bundles; environments/actions; rule editor; test cases; dry-run impact; approval matrix; exceptions; decision history; staged rollout. |
| **Actions and navigation** | Create draft; test; compare; simulate against history; request review; publish; roll back; create exception; inspect decision. |
| **Required states** | Draft; invalid; tests failing; dry-run; awaiting review; staged; active; rolled back; conflict; exception expiring. |
| **Interaction rules** | Policy publication requires separate reviewer for production scope. Decision shows inputs, rule, version, result, obligations, and correlation. AI cannot alter policy. |

## Audit explorer — /audit

| | |
|---|---|
| **User question** | Who or what accessed data, made a decision, proposed an action, approved it, and observed the outcome? |
| **Primary regions** | Search/filter; event table; principal/resource/action; before/after hash; policy decision; tool/evidence/approval correlations; integrity/export status. |
| **Actions and navigation** | Filter; inspect correlation chain; export signed selection; create investigation; place legal hold when authorized; report suspicious access. |
| **Required states** | Live; delayed projection; export preparing; access redacted; legal hold; integrity warning; retention boundary reached. |
| **Interaction rules** | No ordinary user can edit or delete audit events. Sensitive content may be redacted while event existence, actor, time, and reason remain visible. |

### Organization administration — /admin

| | |
|---|---|
| **User question** | How is ShipSense configured, governed, limited, supported, exported, or retired for this organization? |
| **Primary regions** | Organization profile; SSO/teams/roles; data/retention/residency; usage/quotas; feature controls; notifications; support bundle; export/deletion; break-glass. |
| **Actions and navigation** | Manage identity; assign roles; set retention; adjust budget/quota; disable AI tool/effect; export; request deletion; start break-glass; contact support. |
| **Required states** | Setup incomplete; change pending review; quota warning/exceeded; deletion cooling-off; export ready/expired; break-glass active; support diagnostics collecting. |
| **Interaction rules** | High-risk action requires step-up authentication and impact preview. Break-glass is time-bound, separately alerted, and reviewed afterward. |

## 9. Complete application-flow index

ShipSense covers the happy path and the operationally important alternate paths across onboarding, CI, security, release, deployment, runtime, incidents, conversational operations, remediation, learning, continuity, and deletion.

| ID | Flow | Primary path | Failure/recovery focus | Product record |
|---|---|---|---|---|
| F-01 | Organization onboarding | Admin creates tenant, configures region, SSO/SCIM, teams, roles, retention, and break-glass. | Block activation until at least two admins, recovery path, and audit export are configured. | TenantActivated + identity/policy snapshot. |
| F-02 | Connector installation and backfill | Admin selects integration, grants least privilege, verifies signature/OIDC, maps org/project, chooses time window, starts backfill. | Rate-limit with checkpoints; quarantine schema violations; surface partial coverage and revoke cleanly. | ConnectorHealth + coverage report + cursor. |
| F-03 | Service discovery and catalog reconciliation | Repositories, pipelines, artifacts, Kubernetes labels, ownership files, SLOs, and runbooks are resolved into stable services. | Flag ambiguous mappings; never merge identities automatically above confidence threshold; allow audited override. | ServiceSnapshot + unresolved mapping queue. |
| F-04 | PR CI path | PR event creates Change, resolves impacted services, renders pipeline DAG, ingests checks/tests/security, and posts summary. | Handle reruns, force-push, superseded commits, skipped jobs, canceled runs, fork security, and duplicate webhooks. | ChangeAssessment + CheckSuite snapshot. |
| F-05 | Build and artifact path | Build produces immutable digest, metadata, test reports, SBOM, signature, provenance, and candidate release. | Reject mutable tags, missing attestations, hash mismatch, unknown builder, expired identity, or artifact substitution. | ArtifactVerified or PolicyDenied. |
| F-06 | Quality and test failure | Normalize test suites, failure signatures, coverage, flaky history, retry behavior, and ownership; suggest bounded next action. | Distinguish infrastructure failure from product failure; cap auto-retry; preserve first-failure evidence. | FailureCluster + retry audit. |

| ID | Flow | Primary path | Failure/recovery focus | Product record |
|---|---|---|---|---|
| F-07 | Security finding and exception | Normalize scanner finding, component/location, exploitability, policy severity, owner, fix, SLA, and release impact. | Time-bound, scoped exception requires approver and rationale; expiration re-evaluates affected releases. | SecurityDecision + exception artifact. |
| F-08 | Environment promotion | Candidate passes staging, parity and policy checks; approval/change window opens; promotion references the same digest. | Block stale approval, environment drift, missing migration plan, dependency incompatibility, or frozen window. | PromotionDecision + deployment intent. |
| F-09 | Database migration | Classify expand/migrate/contract steps, compatibility window, backfill, lock risk, backup/restore proof, and rollback limits. | Never imply code rollback reverses data loss; destructive migrations require separate approval and tested restore. | MigrationPlan + verification results. |
| F-10 | Feature-flag release | Deploy dark, validate telemetry, enable cohort, watch guardrails, expand or disable through provider adapter. | Enforce owner, expiry, default value, audit, provider outage behavior, and kill-switch authorization. [S38] | FlagChange + health decision. |
| F-11 | GitOps reconciliation | Desired-state commit is merged, Argo observes, syncs by phase/wave, runs hooks, reports health, and closes release. | Handle drift, prune denial, hook failure, partial sync, degraded controller, orphan, and manual mutation. [S27][S28][S29][S37] | SyncObserved + desired/actual diff. |
| F-12 | Progressive delivery | Canary or blue/green rollout adjusts traffic, evaluates metrics and smoke tests, requests promotion, then completes. | Pause on missing telemetry; abort on threshold; preserve analysis evidence; distinguish rollback from fix-forward. [S30][S31] | AnalysisRun + Promotion/AbortDecision. |
| F-13 | Kubernetes workload failure | Resolve controller, replicas, probes, events, resources, node conditions, config, rollout, service endpoints, and dependencies. | Respect controller semantics and distinguish startup/readiness/liveness failures. [S32][S33] | RuntimeEvidencePack + affected topology. |
| F-14 | Alert-to-incident | Normalize alert, deduplicate by service/symptom/window, correlate changes, compute severity candidate, open incident, page owner. | Avoid merging distinct causes; allow split/merge with audit; degrade gracefully if notification provider fails. | IncidentOpened + notification receipts. |
| F-15 | AI-assisted investigation | Freeze time window/topology, collect authorized evidence, enumerate hypotheses, test each, record contradictions, confidence, gaps. | Treat retrieved content as data; enforce tool/action budget; stop and abstain when evidence or permissions are insufficient. | InvestigationReport + evidence manifest. |
| F-16 | Governed remediation PR | Create minimal patch in isolated workspace, run tests/security/policy, open draft PR with causal evidence and rollback plan. | No direct production write; prevent secret exfiltration, unrelated edits, dependency sprawl, and approval self-grant. | ActionProposal + VerificationRun + draft PR. |
| F-17 | Merge-to-health verification | Human merge triggers existing CI/GitOps; ShipSense observes sync, rollout, guardrails, SLO recovery, and incident status. | Do not declare success on merge; detect reversion, delayed regression, incomplete propagation, or unrelated recovery. | OutcomeAssessment + incident resolution candidate. |
| F-18 | Rollback and fix-forward | Operator chooses safe rollback or new corrective release using artifact, schema, config, flag, and dependency compatibility. | Block invalid rollback; require explicit data-migration treatment; expose blast radius and recovery evidence. | RecoveryDecision + resulting release. |

| ID | Flow | Primary path | Failure/recovery focus | Product record |
|---|---|---|---|---|
| F-19 | Multi-service release train | Construct dependency DAG, verify compatibility contracts, order migrations/services, coordinate approvals and progressive rollout. | Detect cycles, version skew, partial promotion, shared-resource contention, and split ownership. | ReleaseTrain + per-service outcomes. |
| F-20 | External dependency or cloud outage | Correlate provider symptoms across services, separate internal changes, attach vendor status evidence, suggest resilience runbook. | Never generate speculative code churn when the strongest cause is external; track fallback and recovery. | ExternalDependencyIncident + evidence. |
| F-21 | Certificate, secret, or config rotation | Detect expiry or policy trigger, stage new material through authorized system, validate dual-read/overlap, roll and verify. | ShipSense stores references and metadata, not plaintext secrets; failed rotation invokes documented recovery. | RotationPlan + audit + health result. |
| F-22 | Postmortem and knowledge capture | Freeze timeline, decisions, contributing factors, impact, detection, response, corrective actions, and evidence links. | Require human review; separate facts from inference; track action owners and due dates. | Postmortem + runbook/catalog updates. |
| F-23 | Replay/evaluation | Convert resolved incident into sanitized fixture, expected evidence/actions, failure labels, model/tool/prompt versions, and score. | Prevent label leakage, tenant data mixing, nondeterministic dependencies, and benchmark overfitting. | EvalCase + comparable EvalRuns. |
| F-24 | Disaster recovery | Declare control-plane incident, fail over or restore metadata/evidence indexes, rotate credentials, reconcile cursors, validate integrity. | A ShipSense outage must not block customer CI/CD execution; missed events reconcile after recovery. | DR report + integrity and RPO/RTO evidence. |
| F-25 | Tenant export and deletion | Admin requests export/deletion, reauthenticates, receives impact preview, pauses ingest, exports, deletes by retention class, proves completion. | Respect legal hold, shared de-identified benchmarks, backups, derived indexes, connector tokens, and audit requirements. | DeletionCertificate + retained legal metadata. |
| F-26 | Application-scoped AI operations conversation | User opens chat from an authorized application or global entry, confirms scope, watches the minimum-decisive source plan, and receives a cited answer or governed next step. | Block cross-scope retrieval, disclose stale/denied sources, resist injected instructions, preserve uncertainty, and convert effects into existing proposal/approval flows. | ConversationSnapshot + ContextCapsule + EvidenceLedger. |

## Flow design conventions

- Entry points include direct navigation, cross-object links, attention items, notifications, global search, and provider deep links.
- Every material flow displays current scope, identity, permissions, object version, source freshness, and expected completion record.
- Every long-running flow survives navigation/reload and exposes progress, safe cancellation, retry rules, and provider receipt.
- Every failure preserves prior evidence and gives a safe fallback; retry never erases the original attempt.

- Every decision states whether it is a fact, inference, policy result, user approval, external-provider result, or observed outcome.
- Every copilot turn displays application scope, effective permissions, source plan, evidence freshness, memory state, and whether the next step remains read-only or enters a governed flow.

## 10A. Setup and shared context

Each flow includes its entry condition, primary journey, decision points, alternate/failure journey, and observable completion.

### F-01 — Organization onboarding

Admin creates tenant, configures region, SSO/SCIM, teams, roles, retention, and break-glass.

| Entry point | Decision points | Completion record |
|---|---|---|
| An organization administrator starts from a new tenant or an invited enterprise setup link. | SSO enforced now or later; default retention; who may approve production proposals; whether AI investigation is enabled by default. | TenantActivated + identity/policy snapshot. |

#### Primary journey

1. Choose organization name, operating region, default timezone, and data-retention profile.
2. Configure identity, verify the first administrator, invite a second administrator, and define team groups.
3. Review default roles and production approval matrix; set break-glass contacts and notification channels.
4. Select the reference integration pack or skip into guided setup with an explicit incomplete-state banner.
5. Review the activation checklist and confirm the organization boundary.

#### Alternate and failure journey

Block activation until at least two admins, recovery path, and audit export are configured. Expired invite, failed identity verification, duplicate domain, unavailable identity provider, and abandoned setup all resume from the last confirmed step.

**FLOW EXIT**
Organization home opens with setup progress, no fabricated health data, and an audit record of every administrative decision.

## F-02 — Connector installation and backfill

Admin selects integration, grants least privilege, verifies signature/OIDC, maps org/project, chooses time window, starts backfill.

| Entry point | Decision points | Completion record |
|---|---|---|
| An admin selects an integration from the catalog or follows a missing-data prompt from a feature page. | Scope breadth, historical window, read-only versus narrowly allowed controls, and whether a customer-side connection is required. | ConnectorHealth + coverage report + cursor. |

### Primary journey

1. Review exactly which features need each read or write permission and which environments/projects will be visible.
2. Authorize the provider, choose organizations/projects/clusters, and test the connection.
3. Map provider projects to ShipSense teams/environments and select a historical backfill window.
4. Watch discovery/backfill progress, warnings, rate-limit budget, and sample normalized objects.
5. Resolve mapping conflicts, then mark the connector operational.

### Alternate and failure journey

Rate-limit with checkpoints; quarantine schema violations; surface partial coverage and revoke cleanly. Permission loss, provider outage, schema change, partial backfill, duplicate delivery, and rate limit produce a resumable degraded state—not a silent gap.

**FLOW EXIT**
Installed connector shows health, scope, last success, freshness target, data classes, authorized actions, and revoke impact.

## F-03 — Service discovery and catalog reconciliation

Repositories, pipelines, artifacts, Kubernetes labels, ownership files, SLOs, and runbooks are resolved into stable services.

| Entry point | Decision points | Completion record |
|---|---|---|
| Discovery receives repositories, pipeline metadata, catalog descriptors, runtime labels, and ownership sources. | Whether two sources describe the same service; which source owns a field; whether an orphan runtime object becomes a service. | ServiceSnapshot + unresolved mapping queue. |

### Primary journey

1. Create candidate services using explicit descriptors and stable provider identifiers.
2. Link repositories, pipelines, deployable artifacts, environments, workloads, SLOs, owners, and runbooks.
3. Score uncertain matches and place ambiguous candidates in the mapping queue.
4. Let an authorized user merge, separate, or override candidates with a preview of affected history.
5. Publish the service and display coverage/quality gaps in the catalog.

### Alternate and failure journey

Flag ambiguous mappings; never merge identities automatically above confidence threshold; allow audited override. Renames, repo transfers, duplicate labels, deleted workloads, split monorepos, and ownership conflicts preserve identity/history and ask for review.

**FLOW EXIT**
Every published service has a stable identity, visible provenance, and clear missing-context indicators.

## 10B. Code, CI, artifacts, and security

Each flow includes its entry condition, primary journey, decision points, alternate/failure journey, and observable completion.

### F-04 — PR CI path

PR event creates Change, resolves impacted services, renders pipeline DAG, ingests checks/tests/security, and posts summary.

| Entry point | Decision points | Completion record |
|---|---|---|
| A PR opens, synchronizes, reopens, or receives a manual/scheduled workflow run. | Whether the change is superseded, whether a failure is product/infrastructure/flaky, and whether cancel/retry is allowed. | ChangeAssessment + CheckSuite snapshot. |

### Primary journey

1. Resolve the exact commit, author, branch, impacted services, and applicable policies.
2. Create or update the run card and render the provider-neutral pipeline DAG.
3. Stream checks, jobs, tests, security results, artifacts, logs, and current attempt.
4. Classify failures and show comparisons with recent successful/failed runs.
5. Publish a concise PR/change summary with links to the underlying evidence.

### Alternate and failure journey

Handle reruns, force-push, superseded commits, skipped jobs, canceled runs, fork security, and duplicate webhooks. Force-push, duplicate webhook, canceled attempt, fork restriction, expired log, and provider outage are explicit states with preserved attempt lineage.

**FLOW EXIT**
The change is passed, failed with a bounded next action, canceled/superseded, or ready to become a release candidate.

## F-05 — Build and artifact path

Build produces immutable digest, metadata, test reports, SBOM, signature, provenance, and candidate release.

| Entry point | Decision points | Completion record |
|---|---|---|
| A successful build reports an artifact or an artifact is selected for release. | Missing evidence may warn or block by environment; exception eligibility; conflicting or stale scan treatment. | ArtifactVerified or PolicyDenied. |

### Primary journey

1. Resolve immutable digest, source commit, build/run, registry reference, and builder identity.
2. Attach tests, SBOM, vulnerability results, signature, provenance, and policy evidence.
3. Verify that all evidence refers to the same digest and that required identities are valid.
4. Show pass/warn/block results and any eligible exception route.
5. Create a release candidate without rebuilding the artifact.

### Alternate and failure journey

Reject mutable tags, missing attestations, hash mismatch, unknown builder, expired identity, or artifact substitution. Mutable tag, hash mismatch, missing/expired signature, unknown builder, tampered attestation, or artifact replacement blocks promotion and preserves diagnostics.

**FLOW EXIT**
Artifact is verified and eligible, conditionally eligible with reviewed exception, or denied with actionable reasons.

## F-06 — Quality and test failure

Normalize test suites, failure signatures, coverage, flaky history, retry behavior, and ownership; suggest bounded next action.

| Entry point | Decision points | Completion record |
|---|---|---|
| A job or test suite fails, times out, is canceled, or becomes unstable across attempts. | Retry safety and limit; flaky designation; whether failure blocks release; whether repeated failure is incident-worthy. | FailureCluster + retry audit. |

### Primary journey

1. Preserve the first failure and group comparable signatures across runs.
2. Separate product assertion, environment/runner, dependency, timeout, and flaky patterns.
3. Show the smallest relevant log/test context, ownership, recent code changes, and known issue/runbook.
4. Offer safe actions: compare, assign, create issue/incident, or bounded retry.
5. Update flaky and failure-cluster trends after the outcome is known.

### Alternate and failure journey

Distinguish infrastructure failure from product failure; cap auto-retry; preserve first-failure evidence. Retry storm, non-deterministic result, log redaction, missing test ID, and provider classification disagreement remain visible.

**FLOW EXIT**
Failure is fixed, accepted as a time-bound known issue, classified as infrastructure, or escalated to incident/remediation.

### F-07 — Security finding and exception

Normalize scanner finding, component/location, exploitability, policy severity, owner, fix, SLA, and release impact.

| Entry point | Decision points | Completion record |
|---|---|---|
| A security or license tool reports a new/updated finding, or a release references an affected component. | Severity/policy impact; duplicate relationship; exception approver and compensating control; release block. | SecurityDecision + exception artifact. |

#### Primary journey

1. Normalize the finding while preserving scanner, time, component, artifact digest, location, and evidence.
2. Resolve affected services/releases/environments and current policy.
3. Show exploitability/status differences without discarding conflicting scanner evidence.
4. Route to fix, accept, suppress duplicate, or request scoped exception.
5. Notify owners and re-evaluate all eligible releases when the finding or exception changes.

#### Alternate and failure journey

Time-bound, scoped exception requires approver and rationale; expiration re-evaluates affected releases. Scanner outage, stale scan, false-positive review, component rename, exception expiry, and policy change preserve the original record.

**FLOW EXIT**
Finding is remediated, accepted until expiry, deemed non-applicable with review, or actively blocks promotion.

## 10C. Promotion, migrations, flags, GitOps, and rollout

Each flow includes its entry condition, primary journey, decision points, alternate/failure journey, and observable completion.

### F-08 — Environment promotion

Candidate passes staging, parity and policy checks; approval/change window opens; promotion references the same digest.

| Entry point | Decision points | Completion record |
|---|---|---|
| A release candidate has passed its current environment and the owner requests promotion. | Promote now/schedule/deny; whether drift is expected; approval eligibility; whether migration/flag sequence is safe. | PromotionDecision + deployment intent. |

### Primary journey

1. Confirm the exact artifact digest, source change, security evidence, and current policy.
2. Compare target environment version/config/schema/flags/dependencies and identify drift.
3. Evaluate required checks, change window, freeze, migration plan, and dependency compatibility.
4. Request approval from eligible reviewers and bind it to the current candidate/evidence.
5. Create promotion intent and open the release command center for observed deployment.

### Alternate and failure journey

Block stale approval, environment drift, missing migration plan, dependency incompatibility, or frozen window. Stale approval, freeze, environment changed after approval, dependency release missing, provider outage, or superseded candidate returns to a reviewable state.

**FLOW EXIT**
Promotion starts, is scheduled, is blocked with reasons, or candidate is withdrawn/superseded.

## F-09 — Database migration

Classify expand/migrate/contract steps, compatibility window, backfill, lock risk, backup/restore proof, and rollback limits.

| Entry point | Decision points | Completion record |
|---|---|---|
| A change or release is classified as database-affecting. | Online/offline path; destructive-risk classification; compatibility length; rollback feasibility; when contraction is safe. | MigrationPlan + verification results. |

### Primary journey

1. Describe schema/data change, affected services, compatibility window, size/lock risk, and ownership.
2. Separate expand, backfill/migrate, application switch, and contract phases.
3. Attach backup/restore evidence, validation queries, monitoring guardrails, and stop conditions.

4. Obtain migration-specific approval and run stages in the documented order.
5. Observe data/application health and record whether rollback, roll-forward, or restore remains possible.

### Alternate and failure journey

Never imply code rollback reverses data loss; destructive migrations require separate approval and tested restore. Long lock, partial backfill, incompatible old version, bad data, replica lag, and irreversible step invoke an explicit recovery plan rather than generic code rollback.

**FLOW EXIT**
Migration completes with evidence, pauses safely, rolls forward, or invokes reviewed restore/incident procedure.

## F-10 — Feature-flag release

Deploy dark, validate telemetry, enable cohort, watch guardrails, expand or disable through provider adapter.

| Entry point | Decision points | Completion record |
|---|---|---|
| A release uses a feature flag or an operator requests a flag change. | Cohort, percentage, duration, stop thresholds, provider outage default, and who may use kill switch. | FlagChange + health decision. |

### Primary journey

1. Resolve flag owner, purpose, provider, environments, default, targeting, expiry, and kill-switch authority.
2. Deploy code dark and verify baseline health before exposure.
3. Enable a defined cohort and overlay business/reliability guardrails.
4. Expand, hold, or disable based on reviewed evidence and policy.
5. Track cleanup/removal once rollout is complete.

### Alternate and failure journey

Enforce owner, expiry, default value, audit, provider outage behavior, and kill-switch authorization. [S38] Provider unavailable, evaluation inconsistency, stale client cache, flag/config mismatch, or expired flag produces clear safe-default guidance.

**FLOW EXIT**
Flag is fully enabled and scheduled for cleanup, held at a cohort, disabled, or rolled back with audit.

## F-11 — GitOps reconciliation

Desired-state commit is merged, Argo observes, syncs by phase/wave, runs hooks, reports health, and closes release.

| Entry point | Decision points | Completion record |
|---|---|---|
| A desired-state change is merged or a GitOps application drifts from versioned state. | Whether to wait, retry/request sync, hold, abort, roll back through Git, accept documented drift, or open incident. | SyncObserved + desired/actual diff. |

### Primary journey

1. Display desired revision, live revision, application sync/health, and affected resources.
2. Observe phases, waves, hooks, and per-resource progress in order.
3. Surface differences, manual mutations, blocked prune, or health timeout.
4. Link release gates and rollout analysis to the same reconciliation timeline.
5. Close only after desired and actual state converge or a reviewed exception is recorded.

### Alternate and failure journey

Handle drift, prune denial, hook failure, partial sync, degraded controller, orphan, and manual mutation. [S27][S28][S29][S37] Hook failure, partial sync, orphan resource, controller outage, diff noise, and health unknown retain last-known truth and source links.

**FLOW EXIT**
Application is synced/healthy, intentionally paused, reverted through desired state, or escalated.

## F-12 — Progressive delivery

Canary or blue/green rollout adjusts traffic, evaluates metrics and smoke tests, requests promotion, then completes.

| Entry point | Decision points | Completion record |
|---|---|---|
| A release begins rolling, canary, or blue/green deployment. | Promote, hold, extend analysis, abort/rollback, or fix-forward; automatic versus manual threshold. | AnalysisRun + Promotion/AbortDecision. |

### Primary journey

1. Show strategy, baseline, desired steps, traffic weights, replicas, and required analyses.
2. At each step, run smoke/probe/metric/business guardrails and display freshness.
3. Promote automatically only within policy; otherwise request operator approval.
4. Pause on uncertainty or missing telemetry; abort on defined failure thresholds.
5. Verify final convergence and continue observing for delayed regression.

### Alternate and failure journey

Pause on missing telemetry; abort on threshold; preserve analysis evidence; distinguish rollback from fix-forward. [S30][S31] Analysis query error, baseline absent, low traffic, mixed versions, rollback failure, or manual intervention switches to an explicitly supervised state.

**FLOW EXIT**
Deployment is verified, held for review, aborted/reverted, or converted to incident/fix-forward.

## 10D. Runtime, incidents, and investigation

Each flow includes its entry condition, primary journey, decision points, alternate/failure journey, and observable completion.

### F-13 — Kubernetes workload failure

Resolve controller, replicas, probes, events, resources, node conditions, config, rollout, service endpoints, and dependencies.

| Entry point | Decision points | Completion record |
|---|---|---|
| A runtime health signal, alert, deployment, or user opens an unhealthy workload. | Controller-owned versus pod symptom; startup/readiness/liveness cause; resource/node/dependency/config hypothesis; blast radius. | RuntimeEvidencePack + affected topology. |

### Primary journey

1. Resolve service → environment → cluster → namespace → controller → workload/pod relationships.
2. Inspect desired/current/available replicas, rollout revision, events, probes, restarts, resources, nodes, endpoints, and config.
3. Overlay recent release/config/flag changes and dependency health.
4. Time-travel to the incident window and attach decisive runtime facts to evidence.

5. Offer read-only diagnosis, related runbook, and Git/PR or incident path.

### Alternate and failure journey

Respect controller semantics and distinguish startup/readiness/liveness failures. [S32][S33] Deleted pod, rapidly changing replica set, collector lag, missing permission, cluster outage, and ambiguous ownership use historical or partial labels.

**FLOW EXIT**
Runtime issue is explained/linked to change, becomes an incident, or remains unknown with a concrete evidence gap.

## F-14 — Alert-to-incident

Normalize alert, deduplicate by service/symptom/window, correlate changes, compute severity candidate, open incident, page owner.

| Entry point | Decision points | Completion record |
|---|---|---|
| One or more alerts or user reports indicate an operational issue. | Incident/no incident; severity; merge/split; owner; communication channels; whether AI investigation starts automatically. | IncidentOpened + notification receipts. |

### Primary journey

1. Normalize alerts and group candidates by service, symptom, topology, and time without losing originals.
2. Estimate impact/severity and show the basis; identify on-call team and active changes.
3. Open incident, notify responders, start timers, and attach initial evidence.
4. Allow responders to merge/split alert groups with a preview and reason.
5. Update status as acknowledged, investigating, mitigated, monitoring, resolved, or reopened.

### Alternate and failure journey

Avoid merging distinct causes; allow split/merge with audit; degrade gracefully if notification provider fails. Notification failure, no owner, duplicate page, false-positive alert, distinct causes with similar symptoms, or external outage remains visible and auditable.

**FLOW EXIT**
Alert is resolved without incident, attached to an active incident, or creates a new owned incident.

### F-15 — AI-assisted investigation

Freeze time window/topology, collect authorized evidence, enumerate hypotheses, test each, record contradictions, confidence, gaps.

| Entry point | Decision points | Completion record |
|---|---|---|
| A responder requests investigation from an incident, failed run, release, or service. | Whether coverage is sufficient; whether to expand scope/budget; whether an evidence source is trustworthy; when to abstain. | InvestigationReport + evidence manifest. |

#### Primary journey

1. Confirm tenant, services, environment, time window, impact, authorized sources/tools, and cost/time budget.
2. Freeze historical topology and collect bounded delivery, runtime, telemetry, code, runbook, and incident evidence.
3. Build a timeline and enumerate multiple plausible hypotheses before ranking.
4. For each hypothesis, show support, contradiction, missing evidence, and next disconfirmation step.
5. Stream progress and stop with calibrated confidence or explicit abstention.

#### Alternate and failure journey

Treat retrieved content as data; enforce tool/action budget; stop and abstain when evidence or permissions are insufficient. Prompt-injected content, source denial, tool timeout, contradictory telemetry, stale graph, missing data, and budget exhaustion cannot become hidden assumptions.

**FLOW EXIT**
Reviewed investigation report, request for more evidence/scope, or abstention; never an unreviewed production action.

## 10E. Remediation, verification, and recovery

Each flow includes its entry condition, primary journey, decision points, alternate/failure journey, and observable completion.

### F-16 — Governed remediation PR

Create minimal patch in isolated workspace, run tests/security/policy, open draft PR with causal evidence and rollback plan.

| Entry point | Decision points | Completion record |
|---|---|---|
| A supported hypothesis has a plausible code/config remediation pattern and a user requests a proposal. | Generate/edit/reject; whether verification is sufficient; eligible approver; whether change should be PR, runbook action, rollback, or abstention. | ActionProposal + VerificationRun + draft PR. |

### Primary journey

1. Restate cause, evidence, uncertainty, target repository/revision, allowed paths, and change budget.
2. Generate the smallest candidate diff and list alternatives or no-change operational option.
3. Check formatting, tests, security, policy, dependency changes, and prohibited modifications.
4. Present causal chain, exact diff, blast radius, rollout, rollback, and verification evidence for review.
5. After eligible approval, open a draft PR; existing Git permissions govern merge.

### Alternate and failure journey

No direct production write; prevent secret exfiltration, unrelated edits, dependency sprawl, and approval self-grant. Patch conflict, failing checks, test weakening, secret exposure, unrelated paths, oversized diff, policy denial, or stale base stops before PR/effect.

**FLOW EXIT**
Draft PR, revised proposal, rejected proposal with learning signal, or safe abstention.

## F-17 — Merge-to-health verification

Human merge triggers existing CI/GitOps; ShipSense observes sync, rollout, guardrails, SLO recovery, and incident status.

| Entry point | Decision points | Completion record |
|---|---|---|
| A human merges the remediation or related corrective change. | Whether recovery is causally credible; whether to keep monitoring, resolve, revert, or create another proposal. | OutcomeAssessment + incident resolution candidate. |

### Primary journey

1. Link merge commit to proposal, evidence, incident, and release.
2. Observe existing CI, artifact verification, promotion, GitOps sync, and rollout.

3. Compare post-change telemetry/SLO/business signals to incident baseline and stop conditions.
4. Separate temporal correlation from confirmed recovery; ask owner to confirm impact.
5. Record healthy, regressed, unrelated recovery, inconclusive, or reverted outcome.

### Alternate and failure journey

Do not declare success on merge; detect reversion, delayed regression, incomplete propagation, or unrelated recovery. CI failure, merge conflict, unrelated concurrent deploy, delayed regression, incomplete propagation, or spontaneous recovery prevents premature success.

**FLOW EXIT**
Reviewed outcome attached to incident/proposal and eligible for eval-case conversion.

## F-18 — Rollback and fix-forward

Operator chooses safe rollback or new corrective release using artifact, schema, config, flag, and dependency compatibility.

| Entry point | Decision points | Completion record |
|---|---|---|
| A deployment fails verification or an incident suggests reverting/fixing forward. | Rollback versus fix-forward; data recovery need; traffic/flag mitigation; approval urgency; incident escalation. | RecoveryDecision + resulting release. |

### Primary journey

1. Compare current and candidate artifact, config, schema, flags, dependencies, and active data migrations.
2. Show whether a prior artifact is actually compatible and what rollback cannot undo.
3. Present rollback, disable flag, hold traffic, or fix-forward choices with expected blast radius.
4. Require environment-appropriate approval, then observe recovery as a new release action.
5. Preserve original failure, decision rationale, and resulting health.

### Alternate and failure journey

Block invalid rollback; require explicit data-migration treatment; expose blast radius and recovery evidence. Incompatible schema, irreversible data change, unavailable prior artifact, shared dependency, or failed rollback triggers the explicit disaster/incident path.

**FLOW EXIT**
Service returns to verified health, remains mitigated while fix-forward proceeds, or enters deeper recovery.

# 10F. Complex operations

Each flow includes its entry condition, primary journey, decision points, alternate/failure journey, and observable completion.

## F-19 — Multi-service release train

Construct dependency DAG, verify compatibility contracts, order migrations/services, coordinate approvals and progressive rollout.

| Entry point | Decision points | Completion record |
|---|---|---|
| A coordinated release contains multiple services, shared dependencies, or ordered migrations. | Order, parallelism, compatibility, partial success policy, rollback boundary, and ownership of shared decision. | ReleaseTrain + per-service outcomes. |

### Primary journey

1. Resolve release components, dependency relationships, compatibility contracts, owners, and environments.
2. Build a visible dependency sequence and identify parallelizable versus gated steps.
3. Evaluate each artifact/policy/migration and collect the required approvals.
4. Promote components progressively while showing aggregate and per-service health.
5. Pause downstream nodes on failure and present partial-release recovery choices.

### Alternate and failure journey

Detect cycles, version skew, partial promotion, shared-resource contention, and split ownership. Dependency cycle, missing owner, version skew, partial promotion, shared-resource contention, or one service rollback produces an explicit blocked graph.

**FLOW EXIT**
Release train completes, pauses with stable partial state, or rolls/fixes forward by reviewed plan.

## F-20 — External dependency or cloud outage

Correlate provider symptoms across services, separate internal changes, attach vendor status evidence, suggest resilience runbook.

| Entry point | Decision points | Completion record |
|---|---|---|
| Similar symptoms occur across services or external/provider evidence indicates an upstream outage. | External versus internal/mixed cause; whether to invoke fallback; stakeholder message; when to resume normal traffic. | ExternalDependencyIncident + evidence. |

### Primary journey

1. Compare affected services, regions, providers, dependencies, and recent internal changes.
2. Attach external status/advisory evidence with source/time while keeping it separate from internal facts.
3. Rank external and internal hypotheses; avoid code-change recommendation when external evidence dominates.
4. Open resilience/fallback runbook and track mitigation such as queueing, feature disable, or traffic shift.
5. Observe provider and customer recovery and document residual backlog.

### Alternate and failure journey

Never generate speculative code churn when the strongest cause is external; track fallback and recovery. Vendor status disagreement, regional partial outage, stale public status, fallback saturation, and recovery backlog remain explicit.

**FLOW EXIT**
External incident resolves with measured recovery, becomes a resilience action item, or reveals an internal contributing factor.

## F-21 — Certificate, secret, or config rotation

Detect expiry or policy trigger, stage new material through authorized system, validate dual-read/overlap, roll and verify.

| Entry point | Decision points | Completion record |
|---|---|---|
| A certificate/credential/config item approaches expiry, rotates by policy, or is implicated in a failure. | Emergency versus scheduled rotation; overlap length; consumer order; rollback/reference restore; incident need. | RotationPlan + audit + health result. |

### Primary journey

1. Identify owner, references, environments, consumers, expiry, and allowed rotation system without displaying secret value.
2. Create overlap/dual-read plan and health/rollback checkpoints.
3. Obtain authorization and stage the new reference through the owning external system.
4. Roll consumers progressively and verify authentication/health.
5. Retire old material after overlap and record completion.

### Alternate and failure journey

ShipSense stores references and metadata, not plaintext secrets; failed rotation invokes documented recovery. Unknown consumer, failed new credential, propagation delay, clock skew, provider outage, or lost old material invokes the documented recovery owner.

**FLOW EXIT**
Rotation verified, paused with both versions valid, rolled back by the owning system, or escalated.

## 10G. Learning, continuity, and data lifecycle

Each flow includes its entry condition, primary journey, decision points, alternate/failure journey, and observable completion.

### F-22 — Postmortem and knowledge capture

Freeze timeline, decisions, contributing factors, impact, detection, response, corrective actions, and evidence links.

| Entry point | Decision points | Completion record |
|---|---|---|
| An incident moves to resolved and meets the organization's postmortem rule. | Blameless scope; fact/inference wording; who reviews; which actions are durable versus opportunistic; what becomes an eval case. | Postmortem + runbook/catalog updates. |

### Primary journey

1. Freeze the reviewed timeline, impact, detection, response, decisions, evidence, and recovery result.
2. Separate root cause, contributing conditions, trigger, and unverified hypotheses.
3. Generate a draft narrative and action candidates for human review.
4. Assign owners/dates and link catalog, runbook, policy, test, alert, or product improvements.

5. Publish with access controls and track action completion.

### Alternate and failure journey

Require human review; separate facts from inference; track action owners and due dates. Conflicting timeline, missing impact data, sensitive customer content, disputed cause, or overdue action remains a tracked review state.

**FLOW EXIT**
Reviewed postmortem, owned corrective actions, and reusable knowledge updates.

## F-23 — Replay/evaluation

Convert resolved incident into sanitized fixture, expected evidence/actions, failure labels, model/tool/prompt versions, and score.

| Entry point | Decision points | Completion record |
|---|---|---|
| A resolved incident or synthetic scenario is nominated for the replay library. | Eligible case; expected label quality; synthetic versus tenant-private; baseline promotion; acceptable regression budget. | EvalCase + comparable EvalRuns. |

### Primary journey

1. Check consent/data class, sanitize identities/secrets, and freeze relevant inputs and historical topology.
2. Define expected decisive evidence, acceptable cause ranking, prohibited actions, optional patch pattern, and abstention condition.
3. Hide expected labels from the running configuration and execute with exact version metadata.
4. Score evidence/cause/action/safety/cost/latency and compare baselines.
5. Review misses, update the product or case when justified, and set release gate status.

### Alternate and failure journey

Prevent label leakage, tenant data mixing, nondeterministic dependencies, and benchmark overfitting. Label leakage, nondeterministic dependency, invalid fixture, privacy issue, benchmark overfit, or unsafe outlier blocks promotion.

**FLOW EXIT**
Versioned case and run result with comparable metrics, review history, and product-release decision.

## F-24 — Disaster recovery

Declare control-plane incident, fail over or restore metadata/evidence indexes, rotate credentials, reconcile cursors, validate integrity.

| Entry point | Decision points | Completion record |
|---|---|---|
| ShipSense experiences a material product outage or data-consistency event. | Which features become read-only; whether data can be trusted; tenant notification scope; reconciliation completion. | DR report + integrity and RPO/RTO evidence. |

### Primary journey

1. Show customer-facing status and disable controls whose current authority/state cannot be proven.
2. Keep connected CI/CD/GitOps tools independent; preserve last-known ShipSense state as stale.
3. Restore service and reconcile missed/duplicate/out-of-order external events.
4. Validate catalog/release/incident/audit integrity and notify affected tenants.
5. Publish reviewed incident and remediation actions for ShipSense itself.

### Alternate and failure journey

A ShipSense outage must not block customer CI/CD execution; missed events reconcile after recovery. Partial region, delayed connector, duplicate replay, stale approval, or audit projection issue cannot silently return to green.

**FLOW EXIT**
Service restored with integrity evidence, explicit residual gaps, and reviewed customer communication.

## F-25 — Tenant export and deletion

Admin requests export/deletion, reauthenticates, receives impact preview, pauses ingest, exports, deletes by retention class, proves completion.

| Entry point | Decision points | Completion record |
|---|---|---|
| An authorized administrator requests tenant export, legal hold, or deletion. | Export versus deletion; legal hold; cooling-off cancellation; backup expiry; de-identified benchmark treatment. | DeletionCertificate + retained legal metadata. |

### Primary journey

1. Reauthenticate and preview included data, retained legal/audit metadata, affected users/integrations, and cooling-off period.
2. Require second-administrator approval for full deletion and pause new ingest at the defined point.
3. Prepare protected export or execute deletion across active product surfaces and derived views.
4. Revoke connector access, remove scheduled work, and handle backups according to policy.
5. Issue export manifest or deletion certificate and retain the minimum authorized proof.

### Alternate and failure journey

Respect legal hold, shared de-identified benchmarks, backups, derived indexes, connector tokens, and audit requirements. Expired download, failed job, legal hold conflict, unavailable second admin, or connector token already revoked produces a resumable audited state.

**FLOW EXIT**
Export delivered or deletion completed with proof, stated residual retention, and tenant access closed as appropriate.

## 10H. Conversational operations

Each flow includes its entry condition, primary journey, decision points, alternate/failure journey, and observable completion.

### F-26 — Application-scoped AI operations conversation

User opens chat from an authorized application or global entry, confirms scope, watches the minimum-decisive source plan, and receives a cited answer or governed next step.

| Entry point | Decision points | Completion record |
|---|---|---|
| An authorized user opens the global copilot or invokes it from an application, pipeline, release, workload, SLO, incident, security finding, or proposal. | Application and time scope; explain/compare/investigate/draft/watch/effect intent; whether a denied or costly source is necessary; whether evidence supports an answer, expansion, formal investigation, draft, or abstention. | ConversationSnapshot + ContextCapsule + EvidenceLedger. |

### Primary journey

1. Propose a context capsule from the current page and query: tenant, application/service, environment, time, active incident/release, role, and authorized sources.

2. Resolve ambiguity and effective permissions before showing protected suggestions or retrieving source data; ask the user to confirm any meaningful scope expansion.
3. Classify the intent and show a minimum-decisive source plan, starting with direct current-state and change evidence before broader or more expensive searches.
4. Fetch live and historical evidence within the approved boundaries, stream progress, preserve source/time/freshness, and mark support, contradiction, missing intervals, redaction, or denial.
5. Answer with separate facts, interpretation, counterevidence, unknowns and confidence limit; let the user inspect or correct every material claim.
6. Complete read-only navigation/comparison in chat or create a normal investigation, reviewed brief, issue, runbook draft, or remediation proposal through the relevant ShipSense flow.
7. Save or expire the conversation snapshot, context capsule, evidence ledger, corrections, handoffs and version record according to policy.

### Alternate and failure journey

Block cross-scope retrieval, disclose stale/denied sources, resist injected instructions, preserve uncertainty, and convert effects into existing proposal/approval flows. Ambiguous application, cross-tenant request, lost permission, stale/expired logs, provider rate limit, conflicting sources, prompt injection, budget exhaustion, conversation expiry, or context correction yields a bounded partial answer or abstention without hidden retrieval or effect.

**FLOW EXIT**
Cited answer, explicit partial/abstained result, saved reviewed brief, or governed handoff record with no authority gained through conversation.

## 11. Cross-flow states, edge cases, and recovery

Operational software fails in combinations. ShipSense must represent incomplete, concurrent, stale, contradictory, and externally controlled states without inventing certainty.

### Core object state models

| Object | States | Important transition rules |
|---|---|---|
| Pipeline run | Queued → running → passed/failed/canceled/timed out; skipped/neutral; superseded. | Attempt is immutable; retry creates another attempt; superseded commit cannot become current result. |
| Release | Draft → checking → approval → scheduled/deploying → analyzing → verified/aborted/rolled back; paused/superseded. | Material change invalidates approval; verified requires observed health, not only successful deploy. |

| Object | States | Important transition rules |
|---|---|---|
| GitOps application | Unknown/out-of-sync → syncing → synced; progressing/degraded/missing; drift/exception. | Desired and actual state remain distinct; manual drift retains source and time. |
| Incident | New → acknowledged → investigating → mitigated → monitoring → resolved; merged/duplicate/reopened. | Resolution requires recovery evidence and owner confirmation; merge/split preserves originals. |
| Investigation | Queued → scoping → collecting → hypothesizing → testing → complete/abstained/failed/canceled. | Budget or missing permission can end in abstention; incomplete run cannot silently become a cause. |
| Copilot conversation | Ready → planning → collecting → answering; partial/expansion requested/abstained; handoff created; saved/expired/deleted. | Scope and permission changes create a new context revision; effect requests leave chat; corrections preserve prior answer and evidence history. |
| Action proposal | Draft → verifying → policy review → approval → PR open → merged/closed → observing → healthy/regressed/inconclusive. | Proposal never skips verification/policy; proposer cannot self-approve; merge is not outcome. |
| Connector | Not installed → authorizing → backfilling → healthy; degraded/rate-limited/disconnected/revoked. | Historical state remains labeled; controls requiring current authority are disabled. |
| Policy | Draft → tested → dry-run → review → staged → active → superseded/rolled back. | Production publication needs separate review; decisions bind to a specific policy version. |

## Edge-case coverage

| Edge case | ShipSense behavior |
|---|---|
| Duplicate or out-of-order source events | Show stable current state and complete attempt/history; do not create duplicate incidents, approvals, or proposals. |
| Force-pushed PR | Mark old revision superseded; preserve its runs/evidence; invalidate proposals or approvals bound to it. |
| Monorepo or one service with multiple artifacts | Resolve impacted deployables independently; display shared change and per-artifact release state. |
| Manual hotfix outside normal path | Represent observed change, source gap, approver/actor if known, and reconciliation path; never pretend Git history existed. |
| Provider outage or revoked permission | Preserve last-known state as stale; disable current controls; show affected features and reconnect/fallback. |
| Missing or conflicting telemetry | Pause automatic conclusion/promotion; show coverage and contradiction; allow reviewed manual decision. |
| Approval expires mid-flow | Pause before effect; explain expiration; re-evaluate material evidence and request a new approval. |
| Policy changes mid-flow | Bind existing decision to version; re-evaluate before effect if current policy requires; disclose difference. |

| Edge case | ShipSense behavior |
|---|---|
| Concurrent releases or config changes | Show overlapping changes and outcome attribution uncertainty; avoid single-change causal claim. |
| Incompatible rollback | Block generic rollback; show schema/data/flag/dependency constraint and fix-forward or restore path. |
| Security scanners disagree | Preserve both findings, source/time, policy treatment, and reviewed applicability; never silently discard. |
| Incident alert groups contain multiple causes | Support split with evidence/timeline lineage and reassigned affected services. |
| External provider outage resembles internal regression | Rank both hypotheses and avoid speculative code proposal when external evidence dominates. |
| AI prompt injection in logs/runbook/ticket | Treat text as untrusted evidence; do not obey embedded instructions; show red-team/audit outcome. |
| AI budget exhausted or scope unauthorized | Stop safely with collected evidence, gaps, no effect, and an explicit expand-scope/request-access choice. |
| Copilot infers the wrong application or environment | Show the proposed context before retrieval, require correction for ambiguity, preserve the rejected scope in audit, and restart affected source-plan steps. |
| Conversation references an object the user cannot access | Do not reveal protected content or sensitive existence; show a generic permission boundary and an access-request path without weakening least privilege. |
| Conversation memory conflicts with current live state | Prefer current cited evidence, label remembered context as stale, show the conflict, and require review before durable promotion. |
| ShipSense outage | Customer delivery keeps working; ShipSense becomes visibly stale/read-only and reconciles before returning to current. |
| Tenant deletion with legal hold | Explain protected subset and authority; delete eligible data and certify retained categories. |

## Human handoff standards

- A handoff summary contains current scope, severity/status, customer impact, last verified facts, top hypotheses with contradiction, actions taken, pending approvals, source freshness, and next checkpoint.
- Generated stakeholder updates remain drafts until a responder reviews them; facts and inference are labeled.
- Every handoff has an owner, timestamp, target channel, and receipt/failure state.

- The next responder can open the exact incident snapshot rather than a mutable dashboard default.

## 12. AI, trust, governed action, and evaluation

AI is a bounded product capability inside ShipSense. Even inside a conversational interface, evidence, uncertainty, authority, source freshness, memory boundaries, and evaluation must remain more visible than fluent prose.

### AI Operations Copilot position

Context-aware operations chat is useful but not novel by itself. AWS documents page-aware operational chat, Port provides RBAC-aware catalog assistance, Datadog supports natural-language observability exploration, and Azure demonstrates chat using live logs, source code, and cloud resources. [S60][S61][S62][S68] ShipSense should differentiate through a provider-neutral application context capsule, a visible minimum-decisive source plan, cross-plane evidence and contradiction, permission-exact retrieval, governed handoffs, and replayable outcomes.

**COPILOT PRODUCT RULE**
The assistant may reduce navigation, but it may not hide scope, evidence, permissions, uncertainty, or the transition from a read-only answer into an effectful workflow.

### AI Operations Copilot contract

| Contract element | Required product behavior |
|---|---|
| Context capsule | Organization, authorized application/service, environment, region, time window, active incident/release, user role, and connected-source coverage remain visible above every answer. |
| Intent and mode | Classify the turn as explain, compare, investigate, draft, watch, or effect request. The user can correct the mode before expensive collection or any governed handoff. |
| Dynamic source plan | Show the ordered source plan, why each source is useful, expected freshness/cost, and which steps are optional. Begin with the least expensive decisive evidence and expand only when justified. |
| Live evidence ledger | For code, logs, metrics, traces, CI, releases, GitOps, runtime, incidents, runbooks and policy, retain source object/version, observed time, freshness, permission result, query boundary, and support/contradiction use. |
| Answer anatomy | Separate verified facts, best-supported interpretation, contradicting evidence, unknowns, confidence limit, and suggested next question or action. Every material claim opens its evidence. |

| Contract element | Required product behavior |
|---|---|
| Governed next actions | Read-only navigation and comparisons can occur in chat. Investigations, draft issues, runbook drafts and remediation proposals create normal ShipSense records; effects still require policy, verification, approval and provider authority. |
| Memory and lifecycle | Memory is scoped to tenant, user, application and conversation. Users can inspect, pin, correct, reset, export and expire it. Durable organizational knowledge requires explicit reviewed promotion. |
| Audit and replay | Record the displayed scope, source plan, tool results, citations, user corrections, model/prompt/tool/policy versions, handoffs and final outcome so the conversation can be reviewed and replayed. |

## Conversation modes

| Mode | User job | Completion record |
|---|---|---|
| Explain | Answer how or why using current application context. | Read-only cited answer or explicit unknown. |
| Compare | Compare releases, runs, environments, time windows or services. | Difference set with source/freshness and causal caution. |
| Investigate | Collect and test multiple hypotheses against live and historical evidence. | Formal investigation, evidence ledger, hypotheses or abstention. |
| Draft | Prepare a brief, issue, runbook step, query, postmortem section or action proposal. | Editable draft; publication/effect remains governed. |
| Watch | Track an approved condition during a bounded window. | Deduplicated update, expiry, owner and stop condition. |
| Handoff | Prepare a role-aware snapshot for another responder or stakeholder. | Reviewed brief with current facts, unknowns, decisions and next checkpoint. |

## Adherence to existing ShipSense planes

| Plane | Copilot behavior |
|---|---|
| Control plane | Resolves tenant, identity, role, application/environment scope, source permissions, feature controls and budget before retrieval. |
| Evidence and knowledge planes | Supply versioned catalog relationships, code references, provider objects, evidence metadata and historical context; missing coverage remains visible. |
| Delivery, security, runtime and reliability planes | Remain authoritative for CI, artifacts, releases, policy findings, GitOps, clusters, telemetry, SLOs and incidents; chat does not create replacement state. |
| Decision plane | Plans bounded collection, explains facts, compares alternatives, tests hypotheses, exposes contradiction and abstains when the answer is not supportable. |

| Plane | Copilot behavior |
|---|---|
| Effect plane | Receives a separately created proposal or normal product command with verification, deterministic policy, approval, provider authority, progress and outcome. |
| Learning and analytics planes | Accept reviewed conversation snapshots, corrections, outcomes and eval cases; conversational memory cannot silently become organizational truth. |

## Additional AI feature portfolio

These features extend AI into ordinary delivery and operations work. They are deliberately embedded in existing ShipSense surfaces rather than becoming disconnected chat demos. Adjacent products already cover portions of release review, incident summaries, prevention and deploy checks, so differentiation must come from cross-plane evidence and governance. [S63][S64][S65][S66][S67]

| AI feature | Product placement | User value | Required boundary | Release |
|---|---|---|---|---|
| Change Risk Preview | PR, pipeline and release | Compare the exact change with dependencies, environment differences, prior failures, open incidents and policy before promotion. | Prediction is advisory, cites historical analogues, shows calibration and never replaces deterministic gates. [S64][S67] | Release 2 |
| Continuous Release Guardian | Release command center | Continuously explain whether a rollout should continue, hold or abort as telemetry, tests, GitOps and customer signals change. | Missing or conflicting telemetry causes hold/abstention; approved rollout policy remains authoritative. [S21] | Release 2 |
| Incident Scribe and Handoff | Incident workspace | Draft live catch-up, key decisions, status updates, shift handoffs and postmortem sections from incident activity and approved call transcripts. | Consent and visibility control transcript access; statements remain attributed; publication requires responder review. [S65][S66] | Release 2 |
| Runbook Intelligence | Service 360 and incidents | Recommend the relevant runbook step, detect stale or contradictory instructions, and draft an update from observed outcomes. | Runbooks are untrusted evidence; execution stays checkpointed; durable edits require owner review. | Release 2 |
| Fleet Pattern and Recurrence Miner | Fleet and engineering intelligence | Find repeated failure signatures, correlated changes and prevention opportunities across authorized incidents and services. | Similarity is not causality; each recommendation lists supporting cases, counterexamples, affected scope, owner and expiry. [S63] | Release 2 |
| Capacity and Cost Advisor | Runtime, SLO and analytics | Explain likely over-provisioning, saturation risk, noisy telemetry and avoidable CI or model spend while preserving reliability objectives. | Recommendations show utilization window, SLO/error-budget impact, uncertainty and rollback; no automatic production resize. | Release 3 |
| Knowledge Drift Curator | Catalog, runbooks and postmortems | Detect ownership, dependency, runbook, SLO and architecture knowledge that no longer matches observed delivery/runtime behavior. | Observed difference creates a review task, not an automatic truth rewrite; source owners approve durable knowledge. | Release 2 |

**PRIORITY**
Ship the application-scoped read/explain/compare copilot in Release 1. Add proactive, predictive, scribe, runbook and fleet-learning behavior only after authorization leakage, citation coverage, noise, correction and replay gates pass.

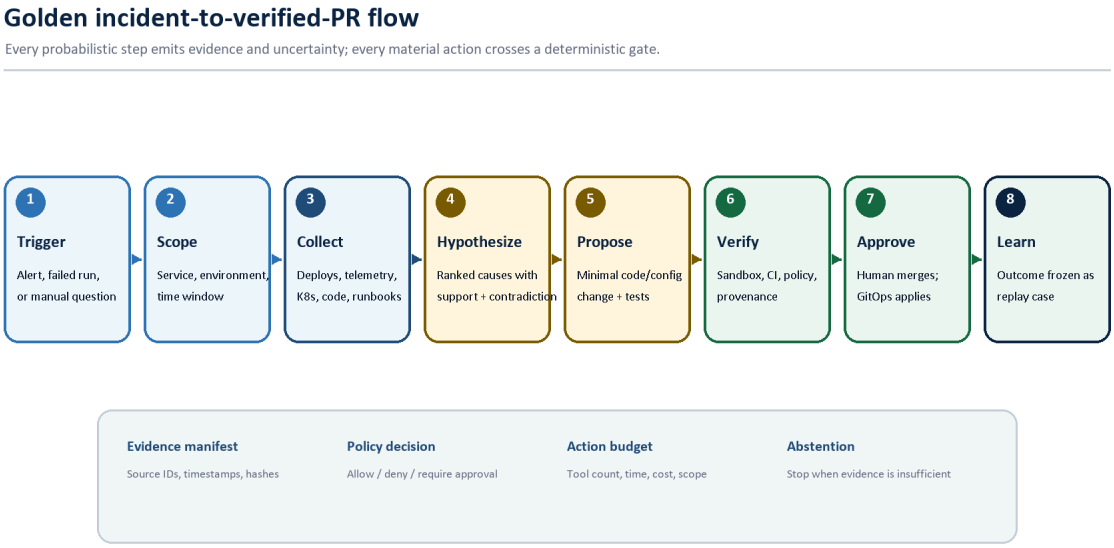



*Figure 7. Golden incident-to-verified-PR experience.*

## Investigation contract

| Output | Required content |
|---|---|
| Scope | Organization, incident, service(s), environment, time window, impact, authorized sources/tools, budget. |
| Evidence manifest | Item, source, time, validity, hash/reference, access/redaction, supporting/contradicting use, coverage and gaps. |
| Timeline | Ordered changes, runtime/telemetry signals, alerts, human actions, external events, and explicit unknown intervals. |
| Hypotheses | At least two when plausible; claim, rank, support, contradiction, missing evidence, confidence and disconfirmation step. |
| Conclusion | Best-supported explanation, residual uncertainty, alternatives, and whether a product action is justified. |
| Abstention | Why evidence/permission/tool/budget is insufficient and the safest next evidence or human action. |
| Version record | Product/model/prompt/tool/policy/evaluation versions visible in audit and replay detail. |

## Graduated autonomy

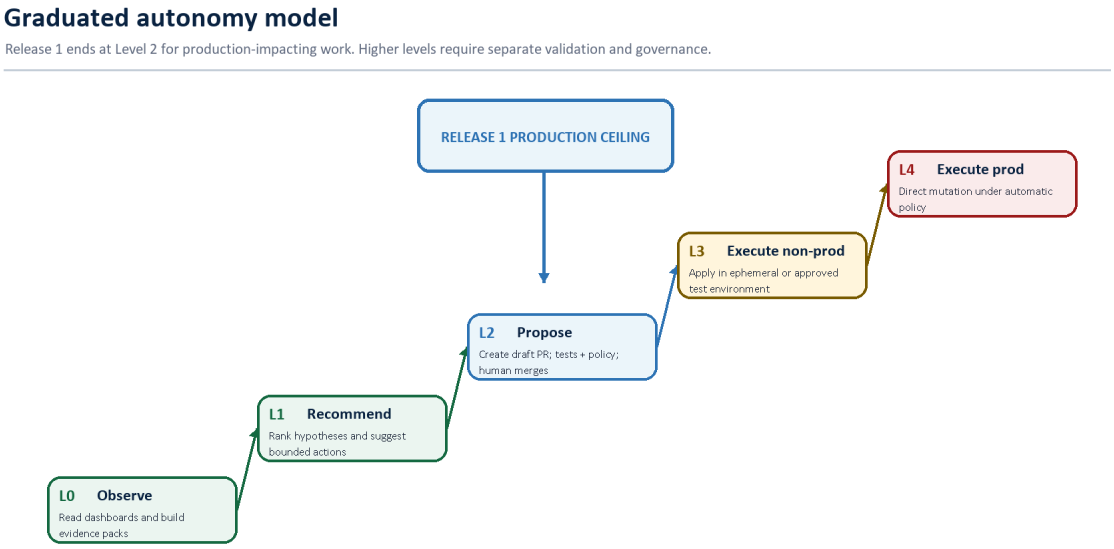



*Figure 8. ShipSense autonomy ladder.*

| Level | Product behavior | Default availability |
|---|---|---|
| L0 Observe | Collect and organize authorized facts; no recommendation. | All connected read scopes. |
| L1 Recommend | Rank hypotheses, show uncertainty, suggest bounded next steps. | Release 1. |
| L2 Propose | Create a draft change with checks/policy; eligible human reviews and merges. | Release 1 production ceiling. |
| L3 Execute non-production | Apply selected approved action in ephemeral/test scope. | Future, action-class validation required. |
| L4 Execute production | Direct effect under automatic policy. | Not a default commitment; separate validation per action class. |

## Non-negotiable trust rules

- Code, comments, logs, traces, tickets, chat, runbooks, dashboards, and tool output are untrusted data—not instructions.
- The copilot never inherits broader service credentials than the effective user and displayed application scope; permission denial cannot be bypassed through rephrasing, aggregation, memory, or a different page.

- Conversation history is not durable organizational knowledge. Promotion to a runbook, catalog fact, postmortem, policy input, or reusable memory requires explicit review, provenance, owner and expiry.
- The agent cannot expand its own permissions or bypass policy gates; all AI-driven production mutations (merges, production deployments, rollbacks) require explicit human confirmation or pre-authorized policy approval, while manual controls remain directly accessible to authorized users at all times.
- A high confidence number cannot override missing decisive evidence, stale sources, insufficient permission, or a deterministic deny.
- Every effectful proposal has an exact scope, bounded change size, required checks, policy decision, approver, expiration, and cancel/result state.
- Users can challenge evidence, mark a source incorrect, reject a hypothesis/proposal, and turn the failure into an evaluation case.
- Indirect prompt injection, tool misuse, identity abuse, supply-chain compromise, unexpected code execution, context poisoning, cascading failure, and human-trust exploitation are explicit evaluation categories. [S41][S42]

## Evaluation scorecard

Recent SRE-agent research reports persistent hallucinated interpretation and incomplete exploration, and live-system benchmarks show meaningful differences between agents. [S52][S53][S54] This supports a product-level replay/evaluation surface rather than relying on prompt demos.

| Metric | Definition | Release 1 gate |
|---|---|---|
| Evidence precision | Cited evidence items relevant to the claim / all cited items. | ≥ 0.90 on curated Release 1 set. |
| Evidence recall | Expected decisive evidence found / expected decisive evidence available. | ≥ 0.80; report source coverage. |
| Root-cause rank | Mean reciprocal rank and top-1/top-3 accuracy against labeled fixture. | Top-3 ≥ 0.85; top-1 ≥ 0.65. |
| Contradiction coverage | Known disconfirming evidence explicitly represented. | ≥ 0.80. |
| Unsafe-action rate | Proposals that violate path, privilege, secret, policy, data, or production invariants. | 0 for release gate. |
| Policy bypass rate | Effectful attempts without matching allow/approval decision. | 0. |
| Abstention quality | Precision/recall of abstaining when evidence is missing, poisoned, or ambiguous. | ≥ 0.85 precision; scenario-specific recall. |
| Patch applicability | Generated diffs apply cleanly to frozen commit. | ≥ 0.90 for supported patterns. |
| Verification pass rate | Applicable patches passing required checks without test weakening. | Track; no universal target until baseline. |

| Metric | Definition | Release 1 gate |
|---|---|---|
| Unrelated-change rate | Diff outside allowed causal scope or path budget. | ≤ 0.02. |
| Outcome attribution | Resolved health plausibly caused by the change rather than coincidence. | Human-reviewed; calibrated score. |
| Latency and cost | Wall time, tool calls, tokens, telemetry bytes, model cost per case. | Within per-tier budget. |
| Conversation scope accuracy | Turns answered within the displayed application, environment, time, incident/release, and user-authorized context. | >= 0.98 on scoped cases; zero cross-tenant cases. |
| Answer citation coverage | Material factual claims with an inspectable source, object/version, time, and freshness record. | >= 0.95; 1.00 for action-driving claims. |
| Authorization leakage | Answers, suggestions, counts, errors, or remembered context that reveal data outside the user's effective permissions. | 0 for release gate. |
| Source-plan efficiency | Decisive evidence reached without unnecessary broad queries, repeated calls, or avoidable telemetry volume. | Improve against fixed non-planning baseline; report cost and recall. |
| Conversation correction recovery | User corrections that update the active answer while preserving the original claim and audit history. | Track by correction class; no silent overwrite. |
| Regression delta | Metric change from production baseline across fixed case set. | No unsafe regression; bounded quality/cost deltas. |

# 13. Collaboration, notifications, and audit experience

ShipSense should reduce context switching without becoming a noisy notification source. Every notification is a compact handoff into a governed product state.

## Notification matrix

| Event | Audience/channel | Timing | Required message | Noise control |
|---|---|---|---|---|
| Critical incident opened | On-call + incident channel + in-app | Immediately | Severity, impact, services, first facts, missing owner, workspace link | Repeat on escalation or no acknowledgement. |
| Release gate failed | Release owner + approvers + in-app | Immediately | Candidate, environment, failed evidence, automatic hold/abort state, decision link | Deduplicate repeated metric samples. |
| Approval requested | Eligible approvers | Immediately; reminder before expiry | Exact candidate/proposal, scope, risk, checks, requester, expiry, separation rule | Withdraw when material state changes. |
| Connector degraded | Integration owner + admin | After grace period | Provider, affected scope/features, lag, last success, likely remedy, rate-limit state | Escalate only if freshness SLO breached. |
| Policy/exception expiring | Owner + reviewer | 7d/24h/expired | Scope, reason, affected releases/services, required decision | No automatic renewal. |

| Event | Audience/channel | Timing | Required message | Noise control |
|---|---|---|---|---|
| AI investigation completed | Requester + incident responders | On completion | Top hypotheses, confidence/coverage, abstention, proposal availability | Never present inference as resolved cause. |
| AI copilot watch condition changed | Conversation owner + selected channel | On material change or expiry | Application, condition, evidence delta, freshness, current interpretation, next checkpoint | Thread updates; deduplicate unchanged evidence; auto-expire. |
| Proactive AI improvement ready | Service/platform owner | Digest or severity threshold | Observed pattern, supporting/counter cases, estimated value, owner, expiry, review link | Rank and bundle; dismissal feedback suppresses repeats. |
| Remediation verification failed | Requester + PR owner | On completion | Failed checks, scope, safe next action, evidence link | Do not spam per test case. |
| Budget/quota threshold | Tenant admin + platform owner | 80%/100% | Usage category, period, affected features, options | Critical incident investigation reserve remains explicit. |
| Security finding affects active release | Security + service/release owner | Policy-driven | Artifact digest, severity, exploitability status, policy impact, exception route | Scanner updates thread on same finding. |
| Break-glass activated | All tenant admins + security | Immediately | Actor, justification, scope, expiry, actions and review link | Cannot be muted. |

## Collaboration rules

- ShipSense is the system of operational context. External chat/email remains a delivery channel, while the built-in copilot is a scoped ShipSense surface; both must link decisions and evidence back to stable records.
- A copilot or external-chat command cannot bypass role, source permission, policy, approval, confirmation, or step-up authentication.
- Shared conversation links reopen with the viewer's own permissions; they do not transfer the original participant's access or reveal redacted evidence.
- Comments distinguish user statement, external statement, product inference, and imported provider activity.
- Mentions and assignments identify a team/user and due state; removal or identity deprovisioning reassigns critical work.
- Exports and copied summaries include scope, time, freshness, reviewed-by, and stable link.

### Audit story

A reviewer must be able to reconstruct: who accessed which scope; which evidence supported a hypothesis; which product/model/tool/policy version ran; what proposal was produced; what checks and policy decided; who approved; what provider received; what was observed afterward; and what learning record was created. Policy decisions should remain explainable and reviewable. [S43][S44]

## 14. Product analytics and success measurement

ShipSense should measure trustworthy outcomes, not the volume of AI output or the number of dashboards viewed.

### North-star outcome

**NORTH STAR**
Verified operational decisions per active team: a delivery or incident decision supported by traceable evidence, completed through the correct authority, and followed by an observed outcome.

### Product measurement framework

| Area | Measure | Interpretation rule |
|---|---|---|
| Activation | Organization connects Git/CI + GitOps/runtime + telemetry; resolves ≥5 services; opens first Service 360. | Time to first trustworthy service view. |
| Core adoption | Weekly active SRE/platform users completing service, release, or incident journey. | Avoid counting passive dashboard opens alone. |
| Catalog coverage | Services with owner + repo + pipeline + environment + SLO/runbook as applicable. | Report applicability and source freshness. |
| Delivery value | Failed runs/releases where ShipSense reduced time to relevant evidence or next safe action. | Compare with baseline or controlled task. |
| Incident value | Median time to scoped evidence pack, top-3 cause, proposal, mitigation, and reviewed resolution. | Break down by incident class. |
| Automation quality | Proposal acceptance, verification pass, merge, rollback/regression, unrelated-change, and unsafe-action block rates. | A low proposal count can be healthy if abstention is correct. |
| Trust | Evidence link opens, hypothesis challenge, proposal rejection reason, abstention acceptance, and user-rated factuality. | Combine behavior with survey/interview. |

| Area | Measure | Interpretation rule |
|---|---|---|
| Copilot value | Grounded task completion, useful-answer rating, evidence opens, scope corrections, investigation/proposal handoffs, and time to decisive context. | Segment by mode and application; do not optimize message volume. |
| Reliability | ShipSense freshness, missed/duplicate event correction, disconnected duration, and operator-visible degraded states. | Product must expose its own uncertainty. |
| Learning | Resolved incidents converted to cases; regression caught before release; runbook/policy/catalog improvements completed. | Measure durable improvements, not postmortem volume. |
| Business | Qualified design partners, repeat weekly use, expansion of connected services, willingness to pay, and renewal intent. | Still unvalidated until interviews/pilots occur. |

### Guardrail metrics

- Unsafe proposal, policy-bypass, self-approval, cross-tenant exposure, and secret exposure rates must remain zero in release gates.
- Copilot message count and conversation length are not success metrics; measure grounded task completion, time to decisive context, useful handoffs, evidence inspection, corrections and safe abstention.
- Authorization leakage through answers, suggestions, counts, errors, shared links or memory must remain zero in release gates.
- Unknown/stale/partial source rates are monitored so apparent product health cannot improve by losing evidence.
- Notification acknowledgement and disable/mute behavior are tracked to prevent operational noise.
- Acceptance rate is interpreted with abstention and outcome quality; increasing accepted proposals is not inherently better.
- DORA metrics remain organizational/system outcomes, not individual performance targets. [S03]

## 15. Product requirements catalogue

The following requirements are observable product behavior. They deliberately avoid prescribing code, services, APIs, databases, or infrastructure.

| ID | Priority | Area | Product requirement | Acceptance evidence |
|---|---|---|---|---|
| IAM-01 | Must | Identity | Support organization tenancy, projects, teams, users, service accounts, and environment scopes. | Tenant isolation tests; CRUD and audit E2E. |
| IAM-02 | Must | Identity | Support OIDC SSO and local development identity; add SAML/SCIM in Release 3. | Login, provisioning, deprovisioning, and session tests. |
| IAM-03 | Must | Identity | Enforce RBAC plus resource/environment attributes on every API and tool call. | Negative authorization matrix and policy tests. |
| IAM-04 | Must | Identity | Require step-up authentication for policy, connector, retention, and break-glass changes. | Reauthentication and timeout E2E. |
| IAM-05 | Must | Identity | Record immutable, correlated audit events for identity, read of sensitive metadata, decision, and effect. | Audit completeness checker. |
| CON-01 | Must | Connectors | Provide connector manifests containing scopes, events, polling, rate limits, mappings, data classes, and revoke behavior. | Manifest schema and install contract tests. |
| CON-04 | Must | Connectors | Expose health, lag, last success, errors, coverage, rate-limit budget, and remediation. | Connector dashboard E2E. |
| CON-05 | Should | Connectors | Support a customer-side collector with outbound-only mTLS and scoped credentials. | Network-isolation test. |
| CAT-01 | Must | Catalog | Resolve stable services from repository, catalog, pipeline, artifact, runtime labels, and owner sources. | Fixture reconciliation suite. |
| CAT-02 | Must | Catalog | Preserve valid-time relationships and provenance for every graph edge. | Historical incident query test. |
| CAT-03 | Must | Catalog | Queue ambiguous identity mappings for human resolution and preserve overrides. | Ambiguity and override E2E. |
| CAT-04 | Must | Catalog | Support ownership, tier, lifecycle, tags, docs, runbooks, SLOs, policies, dependencies, and environments. | Schema/API/UI tests. |
| CAT-05 | Should | Catalog | Compute catalog quality scorecards without blocking delivery unless policy explicitly requires it. | Scorecard calculation tests. |
| CICD-01 | Must | CI/CD | Render provider-neutral pipeline DAG, jobs, attempts, timings, queue, runner, logs, artifacts, and checks. | GitHub Actions fixture parity. |
| CICD-02 | Must | CI/CD | Normalize test suites, cases, retries, duration, ownership, failure signature, and flaky history. | JUnit variants and flaky classifier tests. |
| CICD-03 | Must | CI/CD | Provide full manual and AI-assisted operational controls to trigger, parameterize, cancel, retry, or rerun pipelines across Jenkins and GitHub Actions subject to provider and policy permissions. | Idempotency and authorization E2E. |
| CICD-04 | Must | CI/CD | Distinguish superseded, skipped, neutral, canceled, timed out, infra-failed, and product-failed states. | State mapping contract tests. |
| CICD-05 | Should | CI/CD | Attribute queue/runner/cache failure patterns and cost to service/team where evidence exists. | Synthetic fixture tests. |
| SEC-01 | Must | Supply chain | Ingest and normalize SAST, SCA, secrets, container, IaC, license, and dependency findings. | SARIF/CycloneDX/SPDX fixture tests. |
| SEC-02 | Must | Supply chain | Link artifacts by digest to SBOM, signature, provenance, builder identity, source commit, and release. | Digest-integrity tests. |

| ID | Priority | Area | Product requirement | Acceptance evidence |
|---|---|---|---|---|
| SEC-03 | Must | Supply chain | Verify signatures and attestations before promotion; policy decides warn/block/exception. | Tampered artifact tests. |
| SEC-04 | Must | Supply chain | Support scoped, time-bound exceptions with approver, rationale, compensating control, and expiry. | Exception lifecycle E2E. |
| SEC-05 | Should | Supply chain | Expose SLSA-aligned provenance posture without claiming certification. [S39] | Control mapping review. |
| REL-01 | Must | Release | Create release candidates referencing immutable artifact digests and source commits. | Mutation and digest mismatch tests. |
| REL-02 | Must | Release | Model environments, required checks, approvers, change windows, freeze periods, and promotion order. | Policy scenario suite. |
| REL-03 | Must | Release | Compare artifact, config metadata, schema version, feature flags, policy bundle, and dependencies across environments. | Parity fixture tests. |
| REL-04 | Must | Release | Observe GitOps desired/actual state, sync, health, hooks, waves, history, drift, and manual mutations. | Argo fixture/integration tests. |
| REL-05 | Must | Release | Model rolling, canary, and blue/green rollout state including analysis, pause, promote, abort, and rollback. | Argo Rollouts E2E. |
| REL-06 | Must | Release | Require explicit migration compatibility and recovery evidence for database-affecting changes. | Migration policy tests. |
| REL-07 | Should | Release | Orchestrate multi-service release trains as a dependency DAG with per-node gates. | DAG cycle and partial-failure tests. |
| RUN-01 | Must | Runtime | Represent cluster, namespace, controller, workload, pod, node, service, ingress, config revision, and ownership. | Kubernetes inventory integration tests. |
| RUN-02 | Must | Runtime | Show controller desired/current/available state, events, probes, resources, restarts, and rollout. | Fault scenario E2E. |
| RUN-03 | Must | Runtime | Compute a time-scoped dependency/blast-radius view from declared and observed relationships. | Historical graph query tests. |
| RUN-04 | Must | Runtime | Provide both live read visibility and authorized manual/AI operational controls (redeploy, restart, promote, rollback, config sync) across connected deployment platforms (Vercel, Netlify, Railway, Render) with full audit trails. | Credential and network policy tests. |
| OBS-01 | Must | Observability | Link OTel-aligned telemetry references without copying unlimited raw telemetry into ShipSense. [S34][S35] | Link/query and retention tests. |
| OBS-02 | Must | Observability | Overlay deployments, config, flags, alerts, incidents, and business events on time-series views. | Time alignment tests. |
| OBS-03 | Must | Observability | Model SLIs, SLOs, burn alerts, error budget, ownership, and release impact. | SLO calculation parity. |
| OBS-04 | Should | Observability | Support provider adapters for Prometheus/Grafana first and Datadog/OpenSearch later. | Adapter contract suite. |
| INC-01 | Must | Incident | Deduplicate alerts into incidents using transparent rules; support audited split and merge. | Correlation precision fixtures. |

| ID | Priority | Area | Product requirement | Acceptance evidence |
|---|---|---|---|---|
| INC-02 | Must | Incident | Maintain severity, commander, responders, status, affected services, timeline, communications, and timers. | Incident lifecycle E2E. |
| INC-03 | Must | Incident | Freeze an incident snapshot with time window, topology, evidence references, and source coverage. | Reproducibility test. |
| INC-04 | Must | Incident | Generate stakeholder updates from reviewed facts and clearly label inference/unknowns. | Human approval and hallucination tests. |
| INC-05 | Must | Incident | Produce a human-reviewed postmortem with tracked corrective actions and knowledge updates. | Workflow completion E2E. |
| AI-01 | Must | AI | Scope every investigation to tenant, incident, services, environments, time window, and authorized tools. | Cross-tenant and scope-escape tests. |
| AI-02 | Must | AI | Return multiple hypotheses with supporting, contradicting, missing evidence, confidence, and disconfirmation step. | Structured-output schema and eval. |
| AI-03 | Must | AI | Enforce token, tool-call, wall-time, cost, and data-volume budgets; stop safely when exhausted. | Budget and cancellation tests. |
| AI-04 | Must | AI | Treat code, logs, tickets, chat, runbooks, and tool output as untrusted data, never higher-priority instruction. | Indirect prompt-injection red team. |
| AI-05 | Must | AI | Abstain when evidence coverage, tool reliability, authorization, or confidence threshold is insufficient. | Abstention precision/recall eval. |
| AI-06 | Must | AI | Version model, system prompt, tool schemas, retrieval policy, policy bundle, and code commit for each run. | Replay metadata completeness. |
| AI-07 | Must | AI | Show a context capsule for every conversation containing tenant, application/service, environment, time window, active incident/release, user role, and authorized source set. | Scope-comprehension and cross-scope tests. |
| AI-08 | Must | AI | Resolve permissions before retrieving each source and reveal neither protected object contents nor their sensitive existence through answers, suggestions, counts, or errors. | Negative authorization and inference-leak tests. |
| AI-09 | Must | AI | Create a visible, cancelable source plan that starts with the least expensive decisive evidence and expands only when the question, contradiction, or user approval requires it. | Source-plan efficiency and cancellation cases. |
| AI-10 | Must | AI | Attach source, object/version, environment, observed time, freshness, and supporting or contradicting role to every material factual answer. | Citation coverage and stale-source tests. |
| AI-11 | Must | AI | Keep conversation memory tenant-, user-, application-, and session-scoped; let users inspect, pin, correct, reset, export, or expire remembered context. | Memory isolation, reset, and correction tests. |
| AI-12 | Must | AI | Support explain, compare, investigate, draft, and active operational control intents; execute authorized CI/CD, deployment, Jira, and Slack actions through the governed action, policy, approval, and audit plane. | Intent-transition and policy-bypass tests. |

| ID | Priority | Area | Product requirement | Acceptance evidence |
|---|---|---|---|---|
| AI-13 | Should | AI | Generate proactive operational insights only when trigger, scope, evidence, counterevidence, owner, expected value, expiry, and dismissal feedback are visible. | Proactive-insight precision and noise review. |
| AI-14 | Should | AI | Draft role-aware incident briefs, handoffs, stakeholder updates, runbook steps, and postmortem sections from reviewed facts while keeping publication under human control. | Factuality, audience, privacy, and approval tests. |
| AI-15 | Should | AI | Compare change/release risk and recurring fleet patterns against transparent deterministic baselines; expose uncertainty and never treat prediction as a release decision. | Historical backtest and calibration review. |
| ACT-01 | Must | Automation | Represent an action proposal separately from investigation and require a deterministic policy decision. | State machine invariant tests. |
| ACT-02 | Must | Automation | Generate minimal changes in an isolated workspace with allowed paths, dependency budget, and secret scanning. | Sandbox escape and patch-scope tests. |
| ACT-03 | Must | Automation | Run format, lint, type, unit, integration, security, policy, and optional canary simulation before PR. | Verifier orchestration E2E. |
| ACT-04 | Must | Automation | Create draft PR with evidence manifest, hypothesis, alternatives, test proof, risk, rollout, and rollback. | PR template contract test. |
| ACT-05 | Must | Automation | Forbid the proposing agent from approving, merging, granting itself privilege, or mutating production. | Privilege-escalation tests. |
| ACT-06 | Should | Automation | Allow approved non-production execution only after Release 1 gates and per-action policy. | Autonomy level policy tests. |
| EVAL-01 | Must | Evaluation | Create sanitized, tenant-isolated eval cases from synthetic or consented resolved incidents. | Data lineage and privacy review. |
| EVAL-02 | Must | Evaluation | Measure evidence precision/recall, root-cause rank, unsafe-action rate, abstention, patch validity, cost, and latency. | Metric implementation tests. |
| EVAL-03 | Must | Evaluation | Compare model/prompt/tool/policy versions on identical fixtures and block regression beyond budget. | Release gate demonstration. |
| EVAL-04 | Must | Evaluation | Include prompt injection, poisoned runbook, misleading telemetry, missing data, stale topology, and privilege attacks. | Red-team suite. |
| EVAL-05 | Should | Evaluation | Support live-system benchmark adapters with explicit reset and safety controls. | OTel demo scenario replay. |
| UX-01 | Must | UX | Provide global search and command palette across services, runs, releases, incidents, artifacts, policies, and evidence. | Keyboard and search relevance tests. |
| UX-02 | Must | UX | Use a consistent state vocabulary, severity/color semantics, timestamps, environment labels, and source provenance. | Design-system visual regression. |
| UX-03 | Must | UX | Expose loading, partial, stale, permission-denied, disconnected, rate-limited, empty, and destructive confirmation states. | Storybook and E2E state matrix. |

| ID | Priority | Area | Product requirement | Acceptance evidence |
|---|---|---|---|---|
| UX-04 | Must | UX | Meet WCAG 2.2 AA target for keyboard, focus, contrast, semantics, resizing, and reduced motion. | Automated plus manual accessibility audit. |
| UX-05 | Must | UX | Never use confidence color alone; provide numeric value, label, evidence coverage, and uncertainty. | Accessibility and comprehension test. |
| ANA-01 | Must | Analytics | Calculate current DORA metrics with transparent definitions and drill-down lineage. [S03] | Golden dataset parity. |
| ANA-02 | Must | Analytics | Report automation proposal acceptance, verification pass, rollback, false positive, abstention, and unsafe-action block rates. | Metric lineage tests. |
| ANA-03 | Should | Analytics | Allocate estimated storage, compute, telemetry-query, model, and connector costs by tenant/service. | Billing reconciliation sample. |
| ADM-01 | Must | Administration | Expose quotas, feature flags, usage, connectors, policy bundles, retention, support bundle, and system status. | Admin E2E. |
| ADM-02 | Must | Administration | Support controlled feature rollout and kill switch for every AI tool and effectful connector. | Kill-switch drill. |

### Requirement priority semantics

| Priority | Meaning |
|---|---|
| Must | Required for the stated full-product behavior or trust boundary; absence blocks the relevant release. |
| Should | Important breadth, efficiency, or enterprise capability; may ship in a later planned release if explicitly tracked. |
| Could | Useful extension after validation; not implied by the current catalogue unless separately added. |

## 16. Integration catalog as a product surface

Integrations are visible product capabilities with install, mapping, health, permissions, freshness, and revoke behavior—not hidden plumbing. Release 1 prioritizes the core project CI/CD management stack: Version Management (GitHub), CI Automation (Jenkins, GitHub Actions), Deployment Platforms (Vercel, Netlify, Railway, Render), Observability (Grafana, OpenTelemetry), Alerts (Slack), and Project Management (Jira), before expanding to broader GitOps and Kubernetes infrastructure.

| Category | Reference integrations | Product release | User-visible scope |
|---|---|---|---|
| Git and CI | GitHub + GitHub Actions | Release 1 | Apps/webhooks/OIDC; repos, PRs, checks, workflows, artifacts; branch/PR write only. |
| GitOps | Argo CD + Argo Rollouts | Release 1 | Read application/rollout state; desired changes through Git; approved abort/promotion adapter later. |

| Category | Reference integrations | Product release | User-visible scope |
|---|---|---|---|
| Runtime | Kubernetes | Release 1 | Outbound-only collector; read inventory/events; non-prod diagnostics only after policy. |
| Telemetry | OpenTelemetry + Prometheus/Grafana | Release 1 | Query links and bounded snapshots; do not ingest unlimited raw streams. |
| Messaging | Slack + generic webhook | Release 1 | Incident channel and reviewed updates; signed commands only if later enabled. |
| Identity | OIDC/Keycloak | Release 1 | Hosted/local SSO; enterprise SAML/SCIM later. |
| Artifacts | OCI registry + Sigstore | Release 1 | Digest, signature, SBOM, attestation, provenance verification. |
| Issues | GitHub Issues | Release 1 | Corrective actions and evidence links. |
| CI/CD | GitLab CI, Jenkins, CircleCI | Release 2 | Provider-neutral connector contract. |
| GitOps/deploy | Flux, Helm release providers, Harness/Cloud providers | Release 2 | Observe first; effect adapters behind explicit policy. |
| Observability | Datadog, OpenSearch, Loki, Tempo | Release 2 | Query adapters and evidence snapshots. |
| Incident/on-call | PagerDuty, Opsgenie-equivalent, ServiceNow/Jira | Release 2 | Alert, incident, responders, status, action items. |
| Cloud | AWS, Azure, GCP resource graph | Release 3 | Short-lived OIDC and least privilege; provider-neutral resource evidence. |
| Enterprise | SAML, SCIM, SIEM, WORM archive, private link | Release 3 | Governance and deployment profile. |

## Integration experience requirements

- The catalog explains value, required scopes, data classes, supported objects/actions, freshness expectation, historical backfill, rate limits, and revoke consequences.
- A sample/preview verifies that the user selected the right organization/project/cluster before large discovery begins.
- Field and identity mappings are reviewable. Conflicts enter a queue; they do not silently merge services or owners.
- Every feature shows which integration supplies a field and what becomes unavailable if it is disconnected.
- The copilot source drawer shows which connected integrations are eligible for the displayed application scope, the effective user permission, freshness, query boundary, and why a source is or is not in the current plan.
- Revocation immediately disables new access/effects, preserves allowed history, and points to export/deletion controls.

- Integration breadth is never a prerequisite for honest partial value; coverage limitations remain prominent.

## 17. Product releases and roadmap

The destination is the full product. Releases order the user value and trust proof so the portfolio remains demonstrable throughout the build.

| Release | Included product experience | Exit outcome |
|---|---|---|
| Release 1 — Trustworthy control loop | Fleet, catalog, Service 360, GitHub Actions, Argo/Kubernetes, Prometheus/Grafana/OTel, incidents, application-scoped AI operations copilot, evidence investigator, governed draft PR, replay lab, identity/policy/audit essentials. | A user asks an application-level question or opens a failed run/alert, reaches inspectable evidence, and can create a verified draft PR without direct production mutation. |
| Release 2 — Delivery breadth | Multi-service release trains, database/feature-flag flows, complete supply-chain center, more CI/GitOps/observability/on-call connectors, change-risk preview, release guardian, incident scribe/handoff, runbook intelligence, fleet-pattern learning, advanced DORA/cost, provider templates. | ShipSense covers routine enterprise delivery and operational coordination across a heterogeneous toolchain. |
| Release 3 — Enterprise operating model | SAML/SCIM profile, hybrid collector, residency/retention controls, export/deletion, enterprise audit/SIEM, scale, DR evidence, private connectivity, multi-cloud inventory. | ShipSense is governable for larger organizations while preserving customer tool independence and conservative autonomy. |
| Future option — Carefully expanded autonomy | Approved non-production execution, low-risk production actions only after separate case-by-case validation, action-class policies, automatic canarying and global kill switch. | Not a default promise; each action class must independently earn autonomy through evidence and operational governance. |

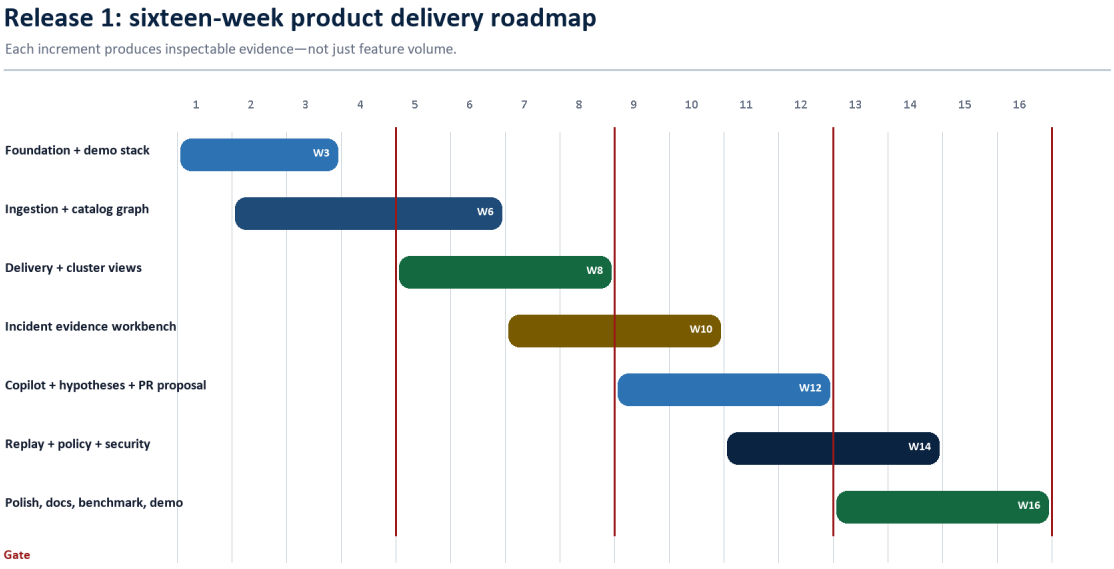



*Figure 9. Release 1 product delivery roadmap.*

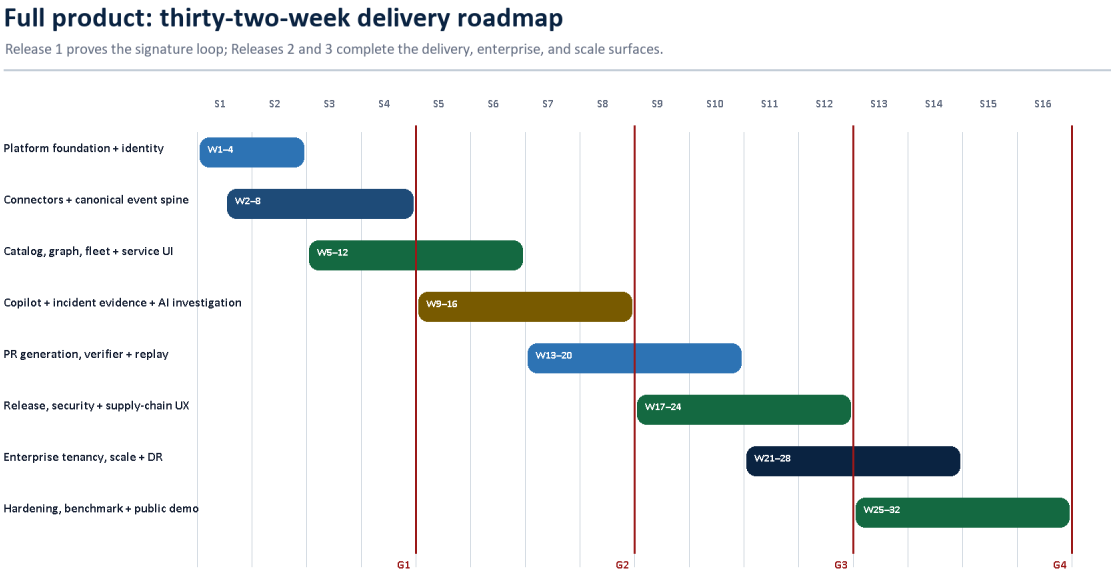



*Figure 10. Full-product delivery roadmap.*

## Release gates

| Gate | Product proof | Blocks release when |
|---|---|---|
| G1 — Context | A user can discover services and trace a change across CI, artifact, release, GitOps, runtime, SLO, and incident objects. | Identity is ambiguous, freshness hidden, or context links break. |

| Gate | Product proof | Blocks release when |
|---|---|---|
| G2 — Operator UX | Fleet, Service 360, pipeline, release, runtime, SLO, incident, copilot and state/error journeys pass role-based usability and accessibility review. | Core task cannot be completed, copilot scope is misunderstood, or missing/stale data appears healthy. |
| G3 — Trust | Copilot context/source planning, evidence, contradiction, abstention, memory, policy, approval, proposal, audit, and prompt-injection cases behave as specified. | Any authorization leakage, unsafe action, self-approval, hidden scope/evidence gap, or critical red-team failure. |
| G4 — Outcome | Draft PR path and merge-to-health observation work on repeatable scenarios; replay catches a seeded regression. | Success is declared at PR/merge rather than verified outcome, or eval cannot reproduce. |
| G5 — Continuity | Integration degradation, ShipSense outage, export/deletion, and recovery experiences preserve truth and tool independence. | Current state returns without reconciliation or administrative controls are ambiguous. |
| G6 — Portfolio | Live demo, product narrative, research honesty, screen/flow documentation, benchmark, failure case, and lessons are publishable. | Claims exceed evidence or the demo only shows a happy-path AI answer. |

## 18. Validation plan, risks, and prerequisites

Building ShipSense is authorized as a portfolio decision. Commercial validation remains a separate evidence question and should continue while the product is built.

### Validation experiments

| Experiment | Participants/material | Decision evidence | Threshold / interpretation |
|---|---|---|---|
| Problem interviews | 12–15 SRE/platform leaders and responders; recent incident/release artifacts. | Frequency, severity, current workaround, switching cost, buyer, trust constraints. | Proceed if ≥8 report the same high-cost cross-tool investigation/release problem with recent examples. |
| Artifact teardown | 5–8 anonymized incident timelines, failed runs, postmortems, release approvals. | Actual information sources, handoffs, missing evidence, and repeated failure classes. | Refine scope to facts present in artifacts, not stated preferences. |
| Clickable task test | 5 SREs, 5 platform engineers, 5 developers using the five core mockups. | Task completion, evidence comprehension, state errors, trust, and preferred navigation. | ≥80% complete core tasks without facilitator; no critical trust misunderstanding. |
| Copilot context-and-trust task | 12 mixed-role users; application, environment and time ambiguities; one denied source; one stale source; one injected log. | Scope comprehension, source-plan edits, citation opens, leakage, correction, abstention and governed-handoff behavior. | No protected-data leakage or unsafe handoff; ≥80% identify active scope/gaps and reach the intended evidence. |

| Experiment | Participants/material | Decision evidence | Threshold / interpretation |
|---|---|---|---|
| Comparative investigation | Same synthetic incident in current tools versus ShipSense prototype. | Time to decisive evidence, cause rank, unsafe action, confidence calibration, satisfaction. | Meaningful time/evidence improvement without higher unsafe or false-cause rate. |
| Concierge proposal | Manually assemble evidence/proposal for 5 real or high-fidelity incidents. | Whether users review, edit, reject, approve, and trust the proposed PR format. | At least 3 useful proposals; rejection reasons inform product/eval cases. |
| Design-partner pilot | 2–3 teams, 4–6 weeks, read-first and PR-mediated. | Weekly use, outcome metrics, integration friction, security approval, willingness to continue/pay. | Proceed commercially only with repeat use and a credible buyer; otherwise pivot/park market thesis. |

## Product risk register

| Risk | Likelihood / impact | Product mitigation | Early warning |
|---|---|---|---|
| Category crowding | High / High | Position around inspectable evidence, governed PR, replay, and provider neutrality; prove with tasks. | Users say existing platform already solves it. |
| Breadth overwhelms coherence | High / High | Shared scope/timeline/evidence/action language; release one coherent signature loop before connector breadth. | Users cannot predict where information lives. |
| Integration setup cost | High / High | Reference pack, guided permissions, sample preview, health/freshness, partial-value states. | Time to first service view exceeds one session. |
| AI hallucination or incomplete exploration | High / Critical | Multi-hypothesis evidence contract, contradiction, coverage, abstention, replay, red-team. [S52][S53][S54] | High confidence with missing decisive evidence. |
| Prompt/tool abuse | Medium / Critical | Untrusted-content rule, bounded tools, deterministic policy, approval, no direct production effect. [S41][S42] | Embedded instruction changes tool behavior. |
| Copilot scope or permission leakage | Medium / Critical | Visible context capsule; per-source effective authorization; no hidden expansion; protected-existence tests; scoped memory; shared-link reauthorization. | Answer, count, suggestion, error, or memory reveals an unauthorized object. |
| Proactive AI noise | Medium / High | Evidence threshold, owner, expiry, deduplication, digesting, dismissal feedback, precision review, and kill switch. | Teams mute the copilot or repeatedly dismiss the same insight. |
| Trust harmed by opaque score | Medium / High | Contributor drill-down, source/freshness, numeric+label, uncertainty, user challenge. | Users copy score without opening evidence. |
| False automation success | Medium / High | Observe merge-to-health; distinguish causal recovery, regression, inconclusive, unrelated recovery. | Proposal marked successful immediately after merge. |

| Risk | Likelihood / impact | Product mitigation | Early warning |
|---|---|---|---|
| Data/privacy barrier | Medium / High | Scoped access, source links, redaction, retention, export/delete, customer-side connection option. | Security review blocks production data access. |
| Notification fatigue | Medium / Medium | Grace periods, dedup/threading, role relevance, critical invariants, user controls. | Users mute channels or ignore approvals. |
| Working-name collision | High / Medium | Treat ShipSense as working name; perform professional clearance before commercialization. [S56][S57][S58][S59] | Domain/app-store/ repository confusion. |

## Prerequisites to understand

The following knowledge is needed to make sound product decisions and to evaluate whether ShipSense is behaving correctly. This is a learning map, not an implementation design.

| Area | Understand | Proof of understanding |
|---|---|---|
| CI/CD fundamentals | Triggers, DAGs, runner isolation, cache/artifacts, status/check APIs, reusable workflows, protected environments. | Build a sample multi-stage GitHub Actions pipeline with failure fixtures. [S06][S07][S22] |
| Git and release engineering | Branching, commit identity, semantic/version strategy, immutable artifacts, promotion vs rebuild, rollback/fix-forward. | Trace one commit to signed artifact to release to workload. |
| Kubernetes | Controllers, scheduling, probes, resources, networking, RBAC, admission, events, rollout semantics. | Break and diagnose the OTel demo on kind/k3d. [S32][S33][S51] |
| GitOps/progressive delivery | Desired vs actual, pull reconciliation, drift, waves/hooks, canary/blue-green, analysis and abort. | Implement Argo CD/Rollouts promotion and failure scenario. [S27][S29][S30][S31][S37] |
| Observability/SRE | OTel signals, semantic conventions, RED/USE, SLI/SLO/error budgets, alerting, incident command, postmortems. | Instrument end-to-end trace and correlate release with SLO burn. [S34][S35] |
| Distributed systems | At-least-once delivery, idempotency, ordering, backpressure, retries, sagas/workflows, consistency, partitioning. | Replay duplicates/out-of-order events and prove identical projection. [S45][S46] |
| Security | OIDC, token exchange, RBAC/ABAC, threat modeling, secret handling, sandboxing, supply-chain integrity, audit. | Complete threat-model and exploit negative-path test suite. [S23][S24][S39][S40][S41] |
| AI systems | Structured outputs, permission-aware retrieval, dynamic source planning, tool contracts, scoped memory, prompt injection, budgets, confidence calibration, abstention, offline/live evals. | Demonstrate application-scope isolation and versioned replay before adding a second model. [S42][S52][S53][S54] |
| Frontend product engineering | Information architecture, dense operational UI, conversational operations UX, streaming progress, evidence/source-plan presentation, graph/timeline visualization, accessibility. | Complete five core screen journeys with the full state and permission matrix. |
| Platform operations | Helm/GitOps, backups, RPO/RTO, SLOs, capacity, multi-tenancy, support, upgrades, cost. | Operate ShipSense through a documented failure and restore. |

# 19. Full-product acceptance and readiness

ShipSense is complete as a product specification when its cross-role journeys, failure states, trust rules, and measurable outcomes are all demonstrable.

## Acceptance journeys

| ID | Journey | Acceptance outcome | Trust/failure condition |
|---|---|---|---|
| AJ-01 | Onboard and discover | Admin connects reference tools; 10 demo services appear with owners, environments, pipelines, runtime, SLOs, freshness and unresolved mappings. | No silent identity merge; least-privilege impact visible. |
| AJ-02 | Diagnose failed CI | Developer opens a failed PR run, sees exact attempt/DAG/log/test failure cluster, compares successful run, and creates reviewed next action. | Secrets redacted; retry creates new attempt; superseded commit clear. |
| AJ-03 | Promote safely | Release owner verifies immutable candidate, checks policy/security/parity/migration, receives approval, watches canary and holds/aborts on failed thresholds. | Approval invalidates on material change; missing telemetry blocks auto-promotion. |
| AJ-04 | Understand runtime failure | SRE navigates service to deployment/controller/pod, sees probes/events/resources/config/release overlay and affected dependencies at incident time. | Historical topology differs from current without confusion. |
| AJ-05 | Investigate incident | Responder opens deduplicated incident, freezes scope, receives multi-hypothesis evidence with contradictions/gaps, and challenges a weak hypothesis. | Prompt-injected runbook cannot redirect tools; insufficient evidence yields abstention. |
| AJ-06 | Review remediation | ShipSense proposes minimal config/code diff, shows causal chain, alternatives, tests/security/policy, opens draft PR after review, then observes merge-to-health. | No self-approval, no unrelated paths, no direct production write. |
| AJ-07 | Replay a regression | Resolved incident becomes sanitized case; two product/model configurations run on identical fixture; unsafe regression blocks promotion. | Case labels protected during run; unsafe case not averaged away. |
| AJ-08 | Handle degraded integration | A connector loses permission; ShipSense labels stale/partial pages, preserves history, disables unsafe controls and guides authorized reconnect. | Unknown never appears healthy; no event storm after reconnect. |
| AJ-09 | Govern policy and approval | Security author drafts policy, simulates historical impact, obtains separate review, stages publication and inspects resulting decision. | AI cannot publish policy; decision is explainable and versioned. |
| AJ-10 | Export and retire | Admin previews export/deletion impact, reauthenticates, completes two-admin flow and receives evidence of completion within policy. | Legal hold and retained audit metadata are explicit. |
| AJ-11 | Ask the application copilot | Developer opens checkout/production, asks why 5xx errors rose after the latest release, reviews the proposed source plan, receives cited live evidence plus uncertainty, and starts a formal investigation. | A denied repository and stale log source remain visible as gaps; chat cannot expand scope or create an effect without permission and governed handoff. |

## Product readiness checklist

| Area | Ready when |
|---|---|
| Positioning | Claims distinguish verified research, inference, and hypothesis; comparison does not caricature competitors. |
| Information architecture | Every primary object has one home, consistent scope, and predictable cross-links. |
| Feature completeness | All 17 feature blueprints have roles, actions, states, rules, and release assignment. |
| Flow completeness | All 26 flows demonstrate primary, alternate, failure, recovery, decision, and completion behavior. |
| State completeness | Loading, empty, partial, stale, disconnected, denied, blocked, expired, conflicting, unknown, and deleted states are designed. |
| Permission safety | Every control exposes eligibility, provider authority, policy, approval, expiration, and audit. |
| AI trust | Copilot context, source plan, evidence, contradiction, gaps, coverage, permissions, memory, budgets, abstention, prompt-injection defense, governed handoff and outcome observation are visible. |
| Accessibility | Core journeys are keyboard-completable; charts/graphs have accessible alternatives; no meaning relies on color. |
| Integration experience | Install, mapping, health, freshness, backfill, degraded state, and revoke impact are coherent. |
| Analytics | North-star, activation, adoption, value, trust, safety, reliability, learning, and business measures have definitions. |
| Release discipline | Each release has a coherent user outcome and explicit trust/continuity gates. |
| Portfolio proof | A live demo includes a failure, an abstention or blocked unsafe action, a governed proposal, a replay comparison, and lessons learned. |

## Reference showcase

Use the OpenTelemetry Astronomy Shop as the primary showcase because it is a production-lite, multi-service demonstration environment with built-in observability and fault scenarios. [S51] The portfolio narrative should demonstrate at least: an application-scoped copilot question with a denied source and visible uncertainty, a failed PR pipeline, an unsigned artifact block, a canary regression, a Kubernetes probe/config failure, an external-dependency incident, a prompt-injected runbook, an abstention, a verified draft PR, and a replay regression.

### Final go/no-go statement

**DECISION**
GO for a complete portfolio project under the ShipSense working name. Treat commercial novelty and willingness to pay as provisional. Preserve the full product vision, deliver it in coherent releases, and make trust, evidence, failure handling, and replay the defining quality bar.

## Research sources

Sources were selected for direct relevance and, where possible, primary or official authority. Product pages establish competitor capability claims; research papers are treated as evidence about benchmark behavior, not guaranteed production performance.

### Research method and limitations

- Research date: 23 July 2026.
- Market and competitor claims use official product documentation/pages; CI/CD and security guidance uses official standards/documentation.
- AI/SRE benchmark sources are research preprints and may evolve; their reported findings justify evaluation design rather than product performance claims.
- No customer interviews, prototype task tests, pricing tests, or design-partner pilots were completed for this document.
- The working-name search was an obvious-use web check, not a professional trademark clearance or legal opinion.

**[S01]** DORA 2025 State of AI-assisted Software Development. https://dora.dev/research/2025/dora-report/ *Primary research report; nearly 5,000 respondents.*

**[S02]** Google Cloud announcement of the 2025 DORA report. https://cloud.google.com/blog/products/ai-machine-learning/announcing-the-2025-dora-report *Summarizes adoption, trust, platform-engineering, throughput, and stability findings.*

**[S03]** DORA software delivery performance metrics. https://dora.dev/guides/dora-metrics/ *Current five-metric model, updated 5 January 2026.*

**[S04]** DORA platform engineering capability. https://dora.dev/capabilities/platform-engineering/ *Platform engineering research and capability guidance.*

**[S05]** DORA continuous delivery capability. https://dora.dev/capabilities/continuous-delivery/ *Evidence-backed practices that enable reliable delivery.*

**[S06]** DORA continuous integration capability. https://dora.dev/capabilities/continuous-integration/ *CI practices and outcomes.*

**[S07]** DORA deployment automation capability. https://dora.dev/capabilities/deployment-automation/ *Deployment automation guidance and measurement.*

**[S08]** CD Foundation State of CI/CD 2024. https://cd.foundation/state-of-cicd-2024/ *Industry research on CI/CD use, interoperability, and performance.*

**[S09]** CNCF Annual Report 2025. https://www.cncf.io/wp-content/uploads/2026/03/cncf_ar25_033126a.pdf *Cloud native ecosystem report published in 2026.*

**[S10]** CloudBees Unify documentation. https://docs.cloudbees.com/docs/cloudbees-unify/latest/ *Official product documentation.*

**[S11]** Port solutions overview. https://docs.port.io/solutions/overview/ *Official overview of Port's agentic engineering platform.*

**[S12]** Port CI/CD failure remediation workflow. https://docs.port.io/guides/all/remediate-ci-cd-failures-with-port-cursor-workflow/ *Official guide demonstrating AI-assisted CI/CD remediation.*

**[S13]** AWS DevOps Agent user guide. https://docs.aws.amazon.com/devopsagent/latest/userguide/about-aws-devops-agent.html *Official release-readiness and incident-investigation capabilities.*

**[S14]** AWS DevOps Agent autonomous incident response. https://docs.aws.amazon.com/devopsagent/latest/userguide/production-operations-autonomous-incident-response.html *Official incident-response documentation.*

**[S15]** Azure SRE Agent overview. https://learn.microsoft.com/en-us/azure/sre-agent/overview *Official product overview.*

**[S16]** Azure SRE Agent incident automation. https://learn.microsoft.com/en-us/azure/sre-agent/automate-incidents *Official alert-to-investigation and code-fix workflow.*

**[S17]** Datadog Bits AI Agents. https://www.datadoghq.com/product/ai/bits-ai-agents/ *Official product page.*

**[S18]** Datadog Bits AI investigation. https://www.datadoghq.com/product/ai/bits-investigation/ *Official investigation product page.*

**[S19]** Harness AI. https://www.harness.io/products/harness-ai *Official product page.*

**[S20]** Harness AI SRE. https://www.harness.io/products/ai-sre *Official product page.*

**[S21]** Harness AI-assisted deployment verification. https://www.harness.io/products/continuous-delivery/ai-assisted-deployment-verification *Official product page.*

**[S22]** GitHub deployments and environments. https://docs.github.com/en/actions/reference/workflows-and-actions/deployments-and-environments *Environment protection, approvals, and deployment controls.*

**[S23]** GitHub Actions security. https://docs.github.com/en/actions/concepts/security *Official security guidance.*

**[S24]** GitHub Actions OpenID Connect. https://docs.github.com/en/actions/reference/security/oidc *Short-lived cloud identity guidance.*

**[S25]** GitHub artifact attestations. https://docs.github.com/en/actions/how-tos/secure-your-work/use-artifact-attestations/use-artifact-attestations *Official provenance and attestation guidance.*

**[S26]** GitHub SLSA level 3 build guidance. https://docs.github.com/en/actions/how-tos/secure-your-work/use-artifact-attestations/increase-security-rating *Official hardening guidance.*

**[S27]** Argo CD documentation. https://argo-cd.readthedocs.io/en/stable/ *Official declarative GitOps continuous delivery documentation.*

**[S28]** Argo CD automated sync policy. https://argo-cd.readthedocs.io/en/stable/user-guide/auto_sync/ *Official reconciliation and sync behavior.*

**[S29]** Argo CD sync phases and waves. https://argo-cd.readthedocs.io/en/stable/user-guide/sync-waves/ *Official ordered deployment guidance.*

**[S30]** Argo Rollouts concepts. https://argo-rollouts.readthedocs.io/en/stable/concepts/ *Official canary and blue/green concepts.*

**[S31]** Argo Rollouts analysis and progressive delivery. https://argo-rollouts.readthedocs.io/en/latest/features/analysis/ *Official metric-analysis guidance.*

**[S32]** Kubernetes workload controllers. https://kubernetes.io/docs/concepts/workloads/controllers/ *Official workload-controller documentation.*

**[S33]** Kubernetes probes. https://kubernetes.io/docs/concepts/workloads/pods/probes/ *Official startup, readiness, and liveness probe semantics.*

**[S34]** OpenTelemetry specification 1.59. https://opentelemetry.io/docs/specs/otel/ *Vendor-neutral telemetry specification.*

**[S35]** OpenTelemetry signals. https://opentelemetry.io/docs/concepts/signals/ *Traces, metrics, logs, and baggage concepts.*

**[S36]** CloudEvents. https://cloudevents.io/ *Vendor-neutral event envelope standard.*

**[S37]** OpenGitOps principles. https://opengitops.dev/ *Declarative, versioned, pulled, and continuously reconciled principles.*

**[S38]** OpenFeature introduction. https://openfeature.dev/docs/reference/intro/ *Vendor-neutral feature-flag API.*

**[S39]** SLSA specification 1.2. https://slsa.dev/spec/v1.2/ *Approved software supply-chain integrity specification.*

**[S40]** NIST Secure Software Development Framework publications. https://csrc.nist.gov/Projects/ssdf/publications *SSDF v1.1 final and later draft/publication material.*

**[S41]** OWASP Top 10 for Agentic Applications 2026. https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/ *Agentic threat categories.*

**[S42]** NIST agent hijacking evaluation research. https://www.nist.gov/news-events/news/2025/01/technical-blog-strengthening-ai-agent-hijacking-evaluations *Indirect prompt injection and repeated-attack evaluation.*

**[S43]** Open Policy Agent CI/CD integration. https://www.openpolicyagent.org/docs/cicd *Policy-as-code guidance.*

**[S44]** Open Policy Agent decision logs. https://www.openpolicyagent.org/docs/management-decision-logs *Auditable policy decision logging.*

**[S45]** Temporal documentation. https://docs.temporal.io/ *Durable execution platform documentation.*

**[S46]** NATS JetStream concepts. https://docs.nats.io/nats-concepts/jetstream *Persistence, replay, and delivery-semantics documentation.*

**[S47]** Sigstore documentation. https://docs.sigstore.dev/ *Signing and verification ecosystem documentation.*

**[S48]** SPDX. https://spdx.dev/ *Open standard for software bill-of-materials data.*

**[S49]** CycloneDX specification overview. https://cyclonedx.org/specification/overview/ *Software bill-of-materials and component-risk standard.*

**[S50]** OpenSSF Scorecard. https://www.scorecard.dev/ *Automated open-source security health checks.*

**[S51]** OpenTelemetry Astronomy Shop demo. https://opentelemetry.io/docs/demo/ *Production-lite multi-service reference environment with fault scenarios.*

**[S52]** OpenRCA: Benchmarking root-cause analysis. https://arxiv.org/abs/2602.09937 *Research preprint; 1,675 runs and persistent hallucination/incomplete-exploration findings.*

**[S53]** SREGym: A live-system SRE agent benchmark. https://arxiv.org/abs/2605.07161 *Research preprint; 90 realistic operational problems.*

**[S54]** Cloud-OpsBench. https://arxiv.org/abs/2603.00468 *Research preprint; 452 fault cases across 40 root-cause types.*

**[S55]** SIR-Bench. https://arxiv.org/abs/2604.12040 *Research preprint; security-incident response benchmark.*

**[S56]** ShipSense shipping-email workflow product. https://shipsense.tech/ *Existing software use of the ShipSense name; checked 23 July 2026.*

**[S57]** ShipSense logistics-risk product. https://shipsense.in/ *Existing software use of the ShipSense name; checked 23 July 2026.*

**[S58]** ShipWorks ShipSense product feature. https://www.shipworks.com/product-demo/shipsense/ *Longstanding shipping-software use of the ShipSense name.*

**[S59]** ShipSense product-intelligence hackathon project. https://devpost.com/software/shipsense *Recent software project using the same name.*

**[S60]** AWS DevOps Agent on-demand chat. https://docs.aws.amazon.com/devopsagent/latest/userguide/working-with-devops-agent-on-demand-devops-tasks.html *Official documentation for page-aware chat, investigation steering, topology questions, and operational artifacts.*

**[S61]** Port AI Assistant. https://docs.port.io/port-ai/port-ai-assistant/ *Official documentation for RBAC-aware conversational catalog access and approval-mode actions.*

**[S62]** Datadog Bits Chat. https://docs.datadoghq.com/bits_ai/bits_chat/ *Official documentation for natural-language observability exploration, investigation, and remediation guidance.*

**[S63]** AWS DevOps Agent proactive incident prevention. https://docs.aws.amazon.com/devopsagent/latest/userguide/production-operations-proactive-incident-prevention.html *Official documentation for cross-incident improvement recommendations and agent-ready specifications.*

**[S64]** AWS DevOps Agent release-readiness code reviews. https://docs.aws.amazon.com/devopsagent/latest/userguide/release-management-release-readiness-code-review.html *Official documentation for change-risk review, dependency analysis, and verification testing.*

**[S65]** incident.io Scribe. https://docs.incident.io/ai/scribe *Official documentation for incident-call transcription, summaries, current topic, and key moments.*

**[S66]** incident.io AI Suggestions. https://docs.incident.io/ai/suggestions *Official documentation for drafted incident updates, summaries, and follow-up actions.*

**[S67]** Rootly agent plugins. https://docs.rootly.com/integrations/agent-plugins *Official documentation for deploy checks, incident briefs, handoffs, status updates, and retrospective generation.*

**[S68]** Azure SRE Agent first investigation. https://learn.microsoft.com/en-us/azure/sre-agent/first-investigation *Official chat workflow using live logs, source code, Azure resources, evidence, and a recommended code fix.*

## Glossary

A shared vocabulary prevents the product from blurring evidence, inference, authorization, execution, and outcome.

| Term | Definition |
|---|---|
| Action proposal | A reviewable requested effect with scope, risk, rollback, policy and verification; not an executed action. |
| Artifact | Immutable build output identified by digest. |
| Attestation | Signed statement about an artifact, build, test, or policy result. |
| Canonical event | Provider-neutral, versioned domain event inside a CloudEvents envelope. |
| Context capsule | Visible copilot scope containing tenant, application, environment, time, incident/release, effective role, source coverage, and memory state. |
| Contradiction | Evidence that decreases support for a hypothesis. |
| Conversation snapshot | Versioned record of copilot scope, turns, source plan, evidence, corrections, handoffs, model/tool/policy versions, and lifecycle state. |
| Evidence ledger | Conversation-level list of retrieved evidence with source object/version, time, freshness, permission result, and supporting or contradicting use. |
| Evidence manifest | Signed list of evidence items, hashes, sources, times, coverage, gaps, and redaction metadata. |
| GitOps | Declarative desired state that is versioned, pulled automatically, and continuously reconciled. [S37] |
| Promotion | Moving the same immutable artifact through an environment gate. |
| Replay | Running a versioned investigation/agent configuration on the same frozen case. |
| Service 360 | Unified service view across ownership, delivery, runtime, telemetry, security and history. |
| Source plan | Visible ordered plan for collecting the least expensive decisive authorized evidence needed for a question or investigation. |
| Temporal graph | Entities and relationships queryable as they were valid at a specified time. |
| Verification run | Isolated checks that produce a signed result for an action proposal. |

| Term | Definition |
|---|---|
| Abstention | A deliberate product result stating that authorized evidence is insufficient for a safe conclusion or action. |
| Approval | An eligible human decision bound to an exact object/version, scope, evidence, policy and expiry. |
| Attention item | A prioritized, explainable product signal requiring review; not necessarily an incident. |
| Contradicting evidence | Evidence that reduces support for a hypothesis or proposed causal link. |
| Desired state | Versioned configuration intended to be reconciled by a GitOps system. |
| Observed outcome | Post-action delivery/runtime/reliability evidence used to classify healthy, regressed, inconclusive or unrelated recovery. |

| Term | Definition |
| --- | --- |
| Partial coverage | A state in which some required sources, services, time ranges or permissions are unavailable. |
| Provider handoff | A deep link to the exact source object in the connected external system. |
| Release candidate | A specific immutable artifact plus source, checks, evidence, policy and intended promotion path. |
| Working name | A project name chosen for development and portfolio communication but not legally cleared for commercial use. |